ZENIKA · BOOTCAMP AI-DRIVEN DEVELOPMENT · MODULE 2

AI-Driven Development

Module 2 : Développement avancé avec l'IA

Faire en équipe : agents personnels, Spec-Driven Development, workflows custom, partage de skills. Août 2026.

Sommaire

1 Boîte à outils d'agents personnels

2 Mémoire, tokens et indexation

3 Spec-Driven Development : fondations

4 Workflow adapté à l'IA

5 Mise en pratique sur un projet

6 Le futur du dev : coder autrement

7 Partage & mutualisation en équipe

© ZENIKA 2026 | All rights reserved

Chapitre 1

Boîte à outils d’agents personnels

Choisir et piloter son harness : outils, droits, skills, hooks.



© ZENIKA 2026 | All rights reserved

Les agents de code en 2026 (1/2)

L'assistant ponctuel devient un **équipier autonome**. Les outils agentiques qui coexistent :

Outil	Paradigme	Point fort
GitHub Copilot	CLI + IDE + app desktop	Skills · hooks · MCP · agents custom · continuité issues/PR/Actions
Claude Code / Anthropic	CLI + IDE + app desktop + web	A popularisé ces primitives (sous-agents, MCP, skills, hooks)
Codex / OpenAI	CLI + IDE + app desktop	Approbation et sandbox pilotés séparément · providers configurables
Vibe / Mistral	CLI	Agentic workflow déclaratif · sous-agents en lecture seule par défaut

© ZENIKA 2026 | All rights reserved

Les agents de code en 2026 (2/2)

Outil	Paradigme	Point fort
OpenCode	CLI	Couplé à OpenRouter, accès multi-modèles
Antigravity / Google	CLI + IDE + Agent Manager	Approche orientée artefacts
IDE-first : Cursor · Devin Desktop *(ex-Windsurf)* · Kiro *(AWS)*	IDE	Composer multi-fichiers · indexation auto du repo · approche specs

i **Hors périmètre** : les outils de *complétion* purs. On parle ici d'outils capables de **planifier, exécuter et itérer** seuls.

© ZENIKA 2026 | All rights reserved

Agentic vs complétion : deux paradigmes

Complétion dans l'IDE

- Suggestion inline au curseur
- L'humain décide et exécute chaque action
- Accès aux outils (terminal, fichiers…) limité
- Contexte limité au fichier ouvert

« Tab pour accepter »

Développement agentique (2026)

- Le dev donne un **objectif**
- L'agent **planifie, exécute, itère** en autonomie
- Accès aux outils : terminal, git, browser, MCP
- Peut déléguer à des **sous-agents** spécialisés

« Fais-le, je valide aux points clés »

i Ce chapitre se concentre sur les outils **agentiques**, et sur comment les **configurer, contextualiser et orchestrer** pour en tirer le meilleur.

© ZENIKA 2026 | All rights reserved

Trois façons de nommer la même boucle

1



ReAct (2022)

Boucle Pensée → Action → Observation.

2



Plan-and-Execute

Planifier la tâche en étapes, puis exécuter et valider.

3



Anthropic aujourd'hui

« gather context → take action → verify work → repeat », pour ses propres agents.

i Les trois décrivent le même mécanisme sous des angles différents : planifier, agir, observer le résultat (tests, erreurs) et réajuster. C'est ce qui distingue « faire » de « compléter ».

© ZENIKA 2026 | All rights reserved

Le harness : ce qui transforme un modèle en agent de code

> *i* **Agent de code = Modèle + Harness.** Le modèle raisonne ; le **harness** lui donne les moyens d'agir : outils, mémoire, permissions, boucle d'exécution.

Ce que fournit un harness, concrètement :

- **Boucle d'outils** : le cycle *observer* → *décider* → *agir* → *observer* qui enchaîne les appels d'outils sans repasser par l'humain à chaque étape.
- **Contexte & mémoire** : ce qui entre dans la context window, ce qui est compacté, ce qui persiste entre sessions (AGENTS.md, mémoire de l'agent : slides suivantes).
- **Permissions & sandboxing** : ce que l'agent a le droit de faire, où, sur quoi.
- **Sous-agents & hooks** : délégation à contexte isolé, automatisation déterministe du cycle de vie.
- **Observabilité** : traces, logs, possibilité de rejouer les décisions prises par l'agent.

> *i* Un **même modèle**, dans **deux harness différents**, peut produire des résultats très différents : le harness est un levier de performance et de coût au moins aussi important que le choix du modèle.

© ZENIKA 2026 | All rights reserved

Panorama des harness actuels (2025-2026) (1/2)

> *i* Tous les agents de code intègrent un harness

Harness	Interface	Modèle(s)	Permissions	Autonomie
GitHub Copilot	Terminal + IDE + app + cloud	Multi-modèle · BYOM	Natives (`--allow-tool` , hooks)	Haute : du mode plan à l'agent cloud
Claude Code	Terminal + IDE + app + web	Claude uniquement	Natives (allow/ask/deny)	Très haute : tourne sans supervision
Aider	Terminal	Multi-modèle (BYOM)	Basiques	Modérée : commit à chaque changement

© ZENIKA 2026 | All rights reserved

Panorama des harness actuels (2025-2026) (2/2)

Harness	Interface	Modèle(s)	Permissions	Autonomie
OpenHands	Docker / API headless	Multi-modèle	Conteneur systématique	Haute : orienté CI/CD
Devin (Cognition)	Cloud intégré	Propriétaire	Sandbox cloud	Très haute : agents concurrents
Pi (earendil-works)	Terminal, RPC embeddable	15+ providers, dont local	Aucune native, déléguée	Configurable

i Ce qui différencie deux harness : interface, permissions, lock-in modèle, gestion du contexte, sous-agents, observabilité.

© ZENIKA 2026 | All rights reserved

Harness et modèle : qui choisit ?

Choisir un harness, choisir un modèle, choisir un fournisseur : **trois décisions désormais séparables**. Trois postures existent sur le marché.

1



Mono-fournisseur

Le harness impose la famille de modèles.

> *Claude Code · Codex*

BILAN

Cohérence maximale, arbitrage impossible.

2



Catalogue multi-éditeur

Un contrat, plusieurs modèles (/model), BYOM côté app.

> *GitHub Copilot*

BILAN

Arbitrage coût / puissance sans changer d'outil.

3



Agnostique par conception

Le fournisseur est un paramètre, pas une décision.

> *OpenCode · Pi · Aider*

BILAN

Liberté totale, aucune garantie fournie.

De ① *le harness décide pour vous* (cohérence, support, un seul contrat) à ③ *vous décidez* (souveraineté, coût maîtrisé, intégration à votre charge).

© ZENIKA 2026 | All rights reserved

La question à poser en entreprise

La question n’est plus « quel outil ? » mais **« quelle posture ? »**.

Elle détermine la **conformité** (où tournent les modèles), le **levier de coût** et la **réversibilité**.

© ZENIKA 2026 | All rights reserved

Deux philosophies de harness : plateforme vs composition

GitHub Copilot, Claude Code : vision **plateforme, descendante**. Un harness intégré à l'assistant, qui décide de la structure (contraintes/coût, contexte, exécution, vérification, cycle de vie). Conviction centrale : *« la différenciation compétitive se déplace du modèle vers le harness qui l'entoure »*. Copilot y ajoute l'ancrage **forge** : le harness s'étend jusqu'aux issues, PRs et Actions.

Pi, OpenCode : vision **primitives-first, ascendante**. Un cœur minimal + extensions, sessions en arbre, multi-fournisseur natif. On **compose** son harness plutôt que d'en adopter un tout fait.

> *i* **« Agentic debt »** : un harness mal entretenu coûte cher dans la durée. À traiter comme un **actif d'équipe partagé et versionné** (hooks, skills, AGENTS.md), pas un réglage individuel jetable.

© ZENIKA 2026 | All rights reserved

Bonnes pratiques d'utilisation d'un harness

- **La policy vit dans le runtime, pas seulement dans le prompt** : une règle de sécurité demandée en langage naturel peut être ignorée par le modèle ; un hook qui bloque l'action, non.
- **Scope minimal** : donner l'accès (fichiers, réseau, commandes) strictement nécessaire à la tâche — jamais « tout, par confort ».
- **Ne jamais désactiver les garde-fous pour aller plus vite** : 45 % du code généré par IA échoue aux tests de sécurité (Veracode, 2025) quand aucun contrôle déterministe ne s'applique.
- **Sandboxer l'exécution** sur les tâches à risque (conteneur, VM, compte dédié) plutôt que de faire confiance à l'agent par défaut.
- **Traiter le harness comme du code d'équipe** : hooks, skills, AGENTS.md versionnés, revus, partagés — pas une config locale que chaque développeur réinvente.

> *i* Ce dernier point rejoint directement les **invariants d'équipe** vus plus loin dans ce module (Ch. 7) : le harness fait partie du **socle commun**.

© ZENIKA 2026 | All rights reserved

Les capacités attendues d’un harness

Le critère de choix n’est plus la marque, c’est la présence de **sept capacités**. Les slides suivantes montrent ce que les harness concrets en font.

- **Terminal *ou* IDE, au choix** : le même harness accessible en CLI, en mode agent dans l’éditeur.
- **Sous-agents à contexte isolé, capables d’écrire** : déléguer un périmètre borné à un agent qui **modifie** vraiment le code. Un sous-agent en lecture seule ne sert qu’à explorer.
- **Branchement d’outils standard** : n’importe quel serveur **MCP**, déclaré dans un fichier versionné.
- **Skills + hooks** : personnaliser le comportement sans modifier le cœur, et automatiser le cycle de vie de façon **déterministe**.
- **Instructions de projet** (`AGENTS.md`) : règles chargées à chaque session, versionnées, partagées par l’équipe.
- **Modes de pilotage** : pouvoir monter l’autonomie **et** borner les droits — les deux slides qui suivent.
- **Scriptable** : mode non interactif, pour la CI et les tâches répétitives.

> *i* Ces sept capacités existaient chez un seul outil en 2025 (**Claude Code**). En 2026 elles sont **la norme** — vérifiez-les avant de choisir, pas la marque.

© ZENIKA 2026 | All rights reserved

Modes de pilotage : monter l'autonomie

Mêmes crans partout, noms différents. L'objectif du module : vous faire monter dans ce tableau.

Cran d'autonomie	GitHub Copilot	Claude Code	Codex
Observer sans agir	mode *plan*	mode *plan*	`sandbox: read-only`
Demander à chaque action	Approbation par défaut	mode par défaut (*ask*)	`approval: untrusted`
Laisser faire dans le périmètre	approbations mémorisées par emplacement	*accept edits*	`on-request` + `workspace-write`
Ne plus rien demander	Contournement des approbations	*bypass permissions*	`approval: never`
Aller au bout seul	Autopilot	auto mode on	*Smart Approvals* (full-auto surveillé)

La cible n'est pas de valider chaque ligne. On monte d'un cran dès qu'on fait confiance au **périmètre**, on redescend quand l'enjeu monte. Autonomie et périmètre de droits sont **deux réglages distincts** → slide suivante.

© ZENIKA 2026 | All rights reserved

Droits d'exécution : où ils se déclarent

Le mode affiché n'est qu'un **défaut** : ce qui décide vraiment, c'est la **liste d'autorisations**, et elle vit dans un fichier.

Harness	Où se déclarent les droits	Exemple
GitHub Copilot	`--allow-tool` / `--deny-tool` au lancement · approbations mémorisées dans `~/.copilot/permissions-config.json` · dossiers de confiance · **tout ouvrir** : `--yolo` (alias de `--allow-all*`)	`--allow-tool 'shell(mvn test)'`
Claude Code	`permissions.allow` / `ask` / `deny` dans `.claude/settings.json` **versionné** (+ local, utilisateur, entreprise)	`"deny": ["Bash(rm:*)"]`
Codex	`~/.codex/config.toml` : `approval_policy` **et** `sandbox_mode`, réglés séparément	`sandbox_mode = "workspace-write"`

Donc oui : en « approbation par défaut » avec une autorisation large en config, l'agent **ne demande plus rien**. Le mode n'est qu'un défaut, la config a le dernier mot.

© ZENIKA 2026 | All rights reserved

Une allowlist n’est pas une sandbox

Une allowlist filtre des **motifs de commande**, pas leurs effets.

`mvn test && curl …`, un script qui en appelle un autre, et le motif est contourné sans être violé.

Le seul périmètre réel est l’**isolation** : conteneur, VM, compte dédié.

© ZENIKA 2026 | All rights reserved

Un concept, plusieurs harness (1/2) : les artefacts

Concept	GitHub Copilot	Claude Code	Standard
Instructions de projet	`.github/copilot-instructions.md` · `AGENTS.md`	`CLAUDE.md` · `AGENTS.md`	`AGENTS.md` (AAIF)
Savoir-faire réutilisable	`.github/skills/<nom>/`	`.claude/skills/<nom>/`	Agent Skills (`SKILL.md`)
Sous-agent à contexte isolé	`.github/agents/*.agent.md`	`.claude/agents/` + outil `Task`	délégation bornée
Automatisation événementielle	`.github/hooks/*.json`	`.claude/settings.json`	hooks de cycle de vie
Branchement d'outils	`mcp-config.json`	`.mcp.json`	MCP ne donne pas de précisions sur sa configuration



Même découpage partout : **règles stables · savoir-faire · automatisation · isolation**. Seuls les chemins changent.

© ZENIKA 2026 | All rights reserved

Un concept, plusieurs harness (2/2) : les actions

Action	GitHub Copilot	Claude Code
Voir sa context window	`/context` · jauge dans VS Code	`/context` · `/usage` (`/cost` en est l'alias)
Compacter · repartir à zéro	`/compact [focus on …]` · `/clear` · `/new`	`/compact` · `/clear` · `/new`
Choisir son modèle	`/model` · BYOM	`/model` · sur modèles Claude
Déléguer à un sous-agent	`/agent` · `copilot --agent <nom>`	outil `Task`
MCP	`/mcp`	`/mcp`
Mode non interactif	`copilot -p "…"`	`claude -p "…"`
Mémoire inter-sessions	Copilot Memory (dépôt + préférences)	auto-memory · outils tiers

Ce qui est vraiment transférable : la **méthode**, *pas toutes* les commandes. Changer de harness coûte un renommage, pas une nouvelle formation.

© ZENIKA 2026 | All rights reserved

AGENTS.md, Skills, Hooks : l'anatomie d'un agent configuré



AGENTS.md

Règles stables, conventions, structure du projet : chargé à chaque session, versionné dans Git.

ACTIVATION

toujours actif



Agent Skills

Savoir-faire réutilisables : SKILL.md + scripts embarqués, chargés quand la tâche s'y prête.

ACTIVATION

à la demande



Hooks

Scripts déclenchés par les événements du cycle de vie (preToolUse, postToolUse, agentStop…).

ACTIVATION

événementiel



Agents custom

Sous-agents à contexte isolé (.github/agents/), parallélisables, périmètre borné.

ACTIVATION

isolation

Règle de base : règles stables → `AGENTS.md` · savoir-faire, workflows longs → Skills · automatisation, scripting → Hooks · périmètre borné → Agents custom.

© ZENIKA 2026 | All rights reserved

« Skill » : même fichier partout, déclenchement différent

Le **format** est un standard ouvert : un dossier, un `SKILL.md`, un frontmatter `name` + `description`. Publié fin 2025, lu par **30+ agents**. Ce qui n'est **pas** standardisé, c'est le **déclenchement** :

Harness	Emplacement projet	Déclenchement
GitHub Copilot	`.github/skills/` *(lit aussi* `.claude/`, `.agents/` *)*	**modèle seul** : sinon *prompt file* `/<nom>` ou `/agent`
Claude Code	`.claude/skills/`	modèle **ou** `/<nom>` : frontière skill / commande floue
Codex	`.codex/skills/` · `~/.codex/skills/`	modèle **ou** `$<nom>` : le `/` reste au contrôle de session

Le fichier est portable, l'action non : ne cherchez pas un `/mon-skill` sur Copilot.

© ZENIKA 2026 | All rights reserved

« Skill » : un dossier, et un mot à trois sens



Un skill n’est pas que du texte

C’est un dossier, qui embarque scripts/ (exécutables), references/ et assets/ à côté du SKILL.md. L’agent lance le script au lieu de réinventer la logique en tokens, et ces fichiers ne coûtent rien tant qu’il ne les ouvre pas. Vrai sur les trois.



« Skill » désigne au moins trois choses

Dans la veille : le fichier (portable), le mécanisme d’activation (propre à chaque harness), et un registre de savoir-faire d’équipe. Vérifiez duquel l’article parle.

© ZENIKA 2026 | All rights reserved

Instructions, prompt, skill, agent : qui déclenche quoi ?

Copilot expose **quatre** mécanismes de personnalisation là où les autres harness en exposent deux ou trois. Le critère qui les sépare n'est pas ce qu'ils contiennent : c'est **qui décide de les charger**.

Mécanisme	Emplacement	Déclenché par	Portée
Instructions	`.github/copilot-instructions.md` · `*.instructions.md`	personne : toujours actif	Le dépôt (ou un glob via `applyTo`)
Prompt file	`.github/prompts/<nom>.prompt.md`	l'utilisateur, par `/<nom>` (dans l'IDE)	Une tâche nommée, répétable
Agent Skill	`.github/skills/<nom>/`	le modèle, sur la `description`	Une capacité, chargée au besoin
Agent custom	`.github/agents/<nom>.agent.md`	l'utilisateur, par `/agent`	Une session entière, avec ses outils

© ZENIKA 2026 | All rights reserved

Le vrai axe de choix : qui décide

Le vrai axe de choix : *toujours* (instructions) → *quand je le demande* (prompt, agent) → *quand c'est pertinent* (skill). Si vous hésitez entre skill et agent, demandez-vous si vous voulez **une tâche** ou **un interlocuteur**.

Le mot « agent » désigne au moins trois choses dans la littérature : l'agent de code lui-même, le sous-agent à contexte isolé, et l'agent custom de Copilot. Vérifiez toujours duquel on parle.

© ZENIKA 2026 | All rights reserved

MCP : brancher l'agent sur son écosystème

Le **Model Context Protocol** (Anthropic, nov. 2024 → Linux Foundation déc. 2025) est le **standard d'intégration** des agents en 2026 : **10 000+ serveurs publics**, adopté par GitHub Copilot, OpenAI, Cursor, Windsurf, Zed…

Catégorie	Serveurs clés pour les développeurs
Dev	`github` · `gitlab`
Documentation	`context7` *(doc à jour des bibliothèques, à la demande)*
Recherche web	`SearXNG` · `brave-search` · `serper`
Projet	`linear` · `jira` · `notion` · `slack`
RAG	`cocoindex` · `project-rag`

Cette liste est un point de départ, pas une recommandation figée : l'écosystème bouge tous les trimestres, et les serveurs pertinents dépendent de la stack du client. À réviser avant chaque session.

© ZENIKA 2026 | All rights reserved

MCP : configurer ses serveurs & outils GitHub

Le **serveur MCP GitHub est pré-installé** dans Copilot CLI, en **lecture seule par défaut** :

```
# vérifier ce qui est exposé
/mcp show github-mcp-server
# élargir un toolset
copilot --add-github-mcp-toolset discussions
# activer l'écriture (issues, PRs)
copilot --enable-all-github-mcp-tools
```

Outils GitHub disponibles : `list_issues` · `create_issue` · `add_issue_comment` · `list_pull_requests` · `create_pull_request` · `search_code` · `get_file_contents` · `list_commits`

© ZENIKA 2026 | All rights reserved

MCP : la config projet, et le cas Claude Code

```
// ./copilot/mcp-config.json : config du projet (versionnée dans Git)
{
  "mcpServers": {
    "linear": { "command": "npx", "args": ["-y", "@linear/mcp-server"],
                "env": { "LINEAR_API_KEY": "${LINEAR_API_KEY}" } }
  }
}
```

i **Sur Claude Code, rien n'est pré-branché** : le serveur GitHub s'ajoute explicitement (`claude mcp add` , ou `.mcp.json` versionné), et **c'est vous qui décidez du périmètre** en choisissant les toolsets exposés et la portée du jeton. Moins pratique, plus lisible : le scope minimal est un choix conscient au lieu d'un défaut à corriger.

© ZENIKA 2026 | All rights reserved

MCP : ce que vous exposez en branchant un serveur

Brancher un serveur MCP, c'est **autoriser un LLM à décider seul quelles données partent chez un tiers**. Trois questions avant chaque branchement :

Question	Ce qu'on vérifie
Qui l'exécute ?	Commande locale (`command`) ou serveur distant (`http`) ? Le local ne sort pas de la machine ; le distant reçoit ce que le modèle choisit de lui envoyer.
Qui l'authentifie ?	Jeton en clair dans le fichier, ou variable d'environnement ? Quels **droits** porte ce jeton : lecture, écriture, suppression ?
Qui le maintient ?	Éditeur officiel ou fork communautaire ? Un serveur non maintenu est un dépôt de secrets ouvert.

© ZENIKA 2026 | All rights reserved

MCP : le précédent, et la réponse d’entreprise

Le précédent à connaître : un serveur MCP Postgres a permis, dès le branchement, l’exécution de **commandes arbitraires** sur la base, suppressions comprises. Le protocole standardise le branchement, **pas** les droits.

La réponse d’entreprise est un registre, pas la vigilance individuelle : un catalogue qui décide quels serveurs sont installables, et par quel transport.

© ZENIKA 2026 | All rights reserved

Outils MCP : élargir GitHub, brancher un tiers

Objectif du fil rouge : repartir avec un écosystème agent personnel opérationnel : outils branchés et bornés, garde-fous déterministes, codebase indexée.

Cette étape : voir ce que l'agent sait déjà faire, ouvrir les droits au strict nécessaire, brancher un serveur tiers, et diagnostiquer une config qui ne marche pas.

Consignes : handout « Lab 1 · Boîte à outils agents personnels », étape 1.1 ; installation : handout « Prérequis des labs ».



© ZENIKA 2026 | All rights reserved

Hooks : automatiser le cycle de vie

Les hooks déclenchent des **scripts déterministes** à des points précis du cycle de vie de l'agent : **la couche de colle** entre l'agent et votre workflow de développement.

Hook	Déclencheur	Cas d'usage
`preToolUse`	Avant chaque appel d'outil	**Bloquer** un `rm -rf` · réécrire une commande verbeuse (rtk)
`postToolUse`	Après un appel d'outil réussi	Formater le fichier écrit · journaliser
`userPromptSubmitted`	À chaque message utilisateur	Enrichir le prompt avec du contexte dynamique
`preCompact`	Avant une compaction	Sauvegarder l'état avant perte d'information
`agentStop`	Quand l'agent a fini sa réponse	Lancer les tests · relire le diff produit
`subagentStop`	Quand un sous-agent termine	Rapatrier les *findings* d'un sous-agent
`sessionEnd`	Fin de session	Mettre à jour AGENTS.md · archiver la trace

© ZENIKA 2026 | All rights reserved

Hooks : l'utilisation des matchers

- **Matcher** : filtre les hooks par **outil visé**. Sans lui, le hook se déclenche sur *tous* les appels.
- **Claude Code** : `matcher` dans `PreToolUse` (ex. `Bash`, `Read|Glob`).
- **Copilot** : `matcher` aussi supporté (regex sur le nom de l'outil), avec commandes `bash` et `powershell` dans la même entrée (`command` en repli multiplateforme).
- **Regex ancrée** : le motif doit correspondre au nom **entier** de l'outil : `bash` ne matche pas `bash_tool`.

> *i* À retenir : les deux supportent `matcher` (récent pour Copilot) ; Copilot ajoute un choix de commande par plateforme (`bash` / `powershell`).

© ZENIKA 2026 | All rights reserved

Hooks : exemple Claude Code

```
// Claude Code : .claude/settings.json — injecter du contexte au démarrage
{
  "hooks": { "SessionStart": [
    { "hooks": [{ "type": "command",
                  "command": "echo 'Convention projet : toute PR référence son numéro de ticket.'" }] }
  ] }
}
```

La sortie standard d'un hook ne s'affiche jamais. Un `echo` n'est utile que là où le harness le **consomme** : c'est le cas de `SessionStart` côté Claude Code, qui injecte la sortie en contexte. Partout ailleurs, on juge un hook à son **effet**, pas à ce qu'il imprime.

© ZENIKA 2026 | All rights reserved

Hooks : exemple Copilot

```
// Copilot : .github/hooks/demo-hook.json — filtrer par outil visé
{
  "version": 1,
  "hooks": { "preToolUse": [
    { "type": "command", "matcher": "bash|view|glob",
      "bash": "$COPILOT_PROJECT_DIR/.github/hooks/check.sh",
      "powershell": "& \"$env:COPILOT_PROJECT_DIR\\.github\\hooks\\check.ps1\"" }
  ] }
}
```

© ZENIKA 2026 | All rights reserved

Lab 1.2 · Boîte à outils agents personnels

Activer un hook local

Objectif : poser un hook `preToolUse` versionné dans le projet, le voir se déclencher, puis le faire refuser un `rm` : la démonstration que « la policy vit dans le runtime, pas dans le prompt ».

Consignes : handout « Lab 1 · Boîte à outils agents personnels », étape 1.2.



© ZENIKA 2026 | All rights reserved

Ce qu’on retient du chapitre 1

L’agent de code en 2026 : les outils agentiques (Copilot, Claude Code, Devin Desktop…) ont remplacé la simple complétion. Ils planifient, exécutent et itèrent de façon autonome.

Anatomie d’un agent configuré : `AGENTS.md` (règles stables) · Agent Skills (savoir-faire) · Hooks (automatisation) · agents custom (isolation). MCP connecte l’agent à l’écosystème. **Ces quatre briques existent chez tous les harness sérieux** : seuls les chemins changent.

Piloter l’autonomie : cinq crans d’approbation, et des droits qui vivent dans un **fichier versionné**, pas dans le mode affiché. La cible est de **superviser**, pas de valider chaque ligne : poussé plus loin au **Ch. 6**.

Hooks : la policy vit dans le runtime : un hook déclenché par le runtime **refuse** ce qu’une consigne d’`AGENTS.md` ne fait que suggérer. Fichier versionné, `matcher` sur l’outil visé, aucune voix au chapitre pour le modèle.

© ZENIKA 2026 | All rights reserved

Chapitre 1 : le fil rouge, et la suite

Fil rouge : le hook posé au LAB 2/2 revient en **Ch. 4** comme **garde-fou de commit** (`SPEC.md` à jour avant de committer), et peut servir comme contrôle de la chaîne de production. Les artefacts de ce chapitre ne sont pas des exercices jetables.

Suite au Ch. 2 : mémoire, tokens et indexation de la codebase : comment donner à l'agent le bon contexte sans saturer sa context window.

© ZENIKA 2026 | All rights reserved

Chapitre 2

Mémoire, tokens et indexation

Ce que l’agent garde d’une session à l’autre, ce qui remplit sa context window, et comment il s’oriente dans un gros dépôt.



© ZENIKA 2026 | All rights reserved

La context window et le KV Cache

À chaque requête, **l'intégralité de l'historique est renvoyée au modèle**, mais le **KV Cache** évite de tout recalculer depuis zéro.

AGENTS.md	Outils MCP	Historique conversation	Nouveau message	Réponse
prompt système	définitions	grandit à chaque échange	~10 tokens	token par token
KV Cache	KV Cache	KV Cache	calcul Q + K + V complet	générée

Le **KV Cache** : vecteurs K et V précalculés, stockés en VRAM, aucun recalcul à chaque requête. Seul le nouveau message est calculé en entier.

> *i* **Temps de calcul : ne s'accumule pas.** Réponses rapides même après 100 échanges. C'est la **VRAM** et la taille du payload réseau qui imposent les limites, pas le recalcul.

© ZENIKA 2026 | All rights reserved

Query, Key, Value : ce que garde le KV Cache

- Chaque token est représenté par 3 vecteurs : **Query** (« que cherché-je ? »), **Key** (« quelle étiquette suis-je ? »), **Value** (« quel sens porté-je ? »). Le KV Cache stocke K et V pour tous les tokens passés : seule la Query des nouveaux tokens est calculée fraîchement.
- Le cache **K+V grossit** à chaque échange et occupe de la VRAM, d'où la limite de context window (de ~128k à ~200k tokens selon le modèle choisi ; ~192k côté Copilot dans VS Code).

© ZENIKA 2026 | All rights reserved

Context window : ce qui s’accumule et ce qui ne s’accumule pas

Dimension	S’accumule ?	Mécanisme	Impact pratique
Temps de calcul	✗ Non	KV Cache : vecteurs K et V réutilisés, seule la Query est calculée sur les nouveaux tokens	Réponses rapides même après 100 échanges
Mémoire (VRAM)	✓ Oui	Le cache K+V grossit à chaque token jusqu’à la limite de context window (~128k à ~200k tokens)	L’IA « oublie » le début → `/compact` pour compresser
Facturation	↘ Réduit	Tokens déjà en cache : jusqu’à −90 % de coût (prompt caching, chez tous les fournisseurs)	Économies sur les sessions longues avec `AGENTS.md` stable, tant que le cache reste chaud
Attention du modèle	Se dégrade	*Lost in the Middle* : sur un long contexte, l’info du milieu est moins bien restituée que le début et la fin	Compacter ne réduit pas que le coût : ça restaure la fiabilité

© ZENIKA 2026 | All rights reserved

« Lost in the Middle »

Même dans la context window, l'attention n'est pas uniforme : **début et fin** sont mieux retenus que le **milieu**.

Un contexte plein ne « tient » donc pas pour autant : placez l'essentiel (objectif, contraintes) **en tête** ou dans `AGENTS.md`, et **compactez** avant que le milieu ne se noie.

© ZENIKA 2026 | All rights reserved

Combien de temps le cache reste-t-il chaud ?

Le KV cache vit **côté serveur**, avec une **durée de vie** : sans activité, il est évincé. Écart d'un facteur **300** entre les modèles. *Valeurs indicatives, pour vous donner une idée : les politiques changent souvent et ne sont pas toutes documentées.*

Fournisseur / modèle	Durée sans activité	Prolongation
Anthropic : Claude	**5 min**, rafraîchi à chaque réutilisation	option **1 h** (écriture 2× au lieu de 1,25×)
OpenAI : GPT-5 → 5.5	**24 h** par défaut depuis mai 2026	*(mode « en mémoire » : 5–10 min, 1 h max)*
OpenAI : GPT-5.6+	**30 min** garanties, parfois plus	seule valeur possible
Google : Gemini 2.5+	cache **implicite** : non documenté	cache **explicite** : **1 h** par défaut, réglable, **stockage facturé**
Bedrock : Claude hébergé	**5 min**	option **1 h** (GA janv. 2026)

© ZENIKA 2026 | All rights reserved

Cache expiré : ce qu’on perd, ce qu’on ne perd pas

Un cache expiré ne perd rien. La conversation est renvoyée **en entier** à chaque requête : vous repayez le préfixe au prix plein et vous attendez plus, mais l’agent n’a **rien oublié**.

La pause déjeuner casse le cache. Avant de partir : **tâche finie →** `/clear` · **tâche en cours →** `/compact`, pour revenir sur un contexte court à reconstruire au lieu d’une session pleine.

© ZENIKA 2026 | All rights reserved

Maîtriser les tokens (1/4) : ce qui entre dans la context window

Un agent autonome consomme **beaucoup de tokens** : comprendre ce qui entre dans la context window (cycle de chargement) évite les mauvaises surprises en milieu de journée.

Élément	Chargé depuis le disque	Dans chaque appel	Ligne `/context`
`AGENTS.md`	1× au démarrage · à sa taille	✓ Prompt système, **caché**	*Custom Instructions*
Définitions d'outils (natifs + MCP)	1× au démarrage · reload via `/mcp`	✓ Toujours envoyé, **caché**	*System Tools* · *MCP Tools*
Historique de conversation	Continu, à chaque échange	✓ Croissant, compacté vers ~80 %	*Messages*
Résultats MCP · fichiers lus	À la demande	✓ Dès l'appel, jusqu'au `/clear`	*Messages*
Agent Skills	Quand le modèle les juge pertinents	✓ Après chargement seulement	*Messages*

© ZENIKA 2026 | All rights reserved

Le piège classique en TP

Tout ceci est lu **une fois au démarrage**, jamais rechargé en cours de session.

Éditer son `AGENTS.md` ne change **rien** avant un `/clear`.

© ZENIKA 2026 | All rights reserved

Tokens (2/4) : mesurer sa context window

LAB INLINE

Une seule question ici : *qu'est-ce qui remplit ma context window ?* Levier de **fiabilité** (cf. *Lost in the Middle*), pas encore de facture.

Mesurez **deux fois** : à froid, puis après 20 minutes de travail.

- Copilot CLI · Claude Code : `/context` · Codex : `/status` (+ indicateur de remplissage en %)
- Comparez les postes : lequel a bougé ? lequel était déjà lourd *avant* que vous ne tapiez quoi que ce soit ?

© ZENIKA 2026 | All rights reserved

Maîtriser les tokens (2/4) : ce que révèle chaque poste

Poste mesuré	Ce que ça révèle
Prompt système · définitions d'outils natifs	le plancher, non compressible
Instructions custom (votre `AGENTS.md`)	vous le payez à **chaque** requête
Outils MCP	souvent plusieurs milliers de tokens par serveur branché
Historique	le seul poste qui grossit vraiment
Espace libre · buffer de réponse	ce qu'il vous reste réellement pour travailler

> *i* Le déclic du chapitre : *« mon `AGENTS.md` et mes serveurs MCP pèsent 15 % de ma context window avant même de commencer ».*

© ZENIKA 2026 | All rights reserved

Maîtriser les tokens (3/4) : réduire ce qui entre

Cinq leviers, du plus immédiat au plus structurant : tous agissent sur ce qu'on vient de mesurer.

Levier	Comment	Gain
Compaction	Auto vers ~80 % ; `/compact`, ou `/compact focus on <sujet>`	Réduit fortement le contexte : résume, ne repart pas de zéro
Nouvelle session	`/clear` vide l'historique, repart de AGENTS.md seul	Zéro accumulation · à préférer quand la tâche est terminée
Prompt caching	Instructions stables entre les requêtes	Jusqu'à 90 % de réduction sur les appels répétés
Agents custom	Déléguer un domaine borné (ex : `src/auth/`)	Context window isolée = pas de pollution du contexte principal
Skills à la demande	Un skill n'entre en contexte que s'il est pertinent	Évite des milliers de tokens de workflow en permanence
Choix du modèle	`/model` : un modèle léger pour le mécanique (attention)	Économie directe, à qualité équivalente

© ZENIKA 2026 | All rights reserved

Maîtriser les tokens (3/4) : deux règles pratiques

Règle pratique : `AGENTS.md` < 500 lignes · un agent custom dès qu'un travail dépasse un périmètre de fichier.

`/model` **en cours de session invalide tout le cache** : changer de modèle change le préfixe, donc l'intégralité de la context window est recalculée et **refacturée**. Le bon moment pour choisir son modèle, c'est **au démarrage**.

© ZENIKA 2026 | All rights reserved

Maîtriser les tokens (4/4) : ce que ça coûte, et où on le lit

Deuxième question, distincte de la première : *combien j'ai dépensé ?* Elle ne se lit **pas** au même endroit, ni dans la même unité.

Harness	Où se lit la context window	Où se lit la dépense
GitHub Copilot	`/context` · jauge IDE	tableau de bord AI Credits, hors session
Claude Code	`/context`	`/cost` en session (`cache_read` / `cache_creation`)
Codex	`/status` · indicateur %	`/usage` en session (activité, quotas)

© ZENIKA 2026 | All rights reserved

Maîtriser les tokens (4/4) : l'entrée pèse le plus

Deux effets opposés sur le prix : la **sortie** coûte ~**5× l'entrée** au token, mais en session l'**entrée domine** largement le volume (fichiers lus, sorties d'outils, historique, prompt système).

Ordre de grandeur par classe de modèle *(relatif, entrée / sortie)* : raisonnement avancé **×5 / ×25** · généraliste **×3 / ×15** · léger **×1 / ×5**.

> *i* **Constat** : c'est l'**entrée** qui pèse le plus sur la facture, malgré son prix unitaire 5× inférieur. Compresser les **entrées** (sortie shell, RAG, `/clear`, `/model` bien choisi) a donc plus d'impact que compresser la **sortie**.

> *i* **Un outil transversal, hors de tout tableau de bord éditeur** : `ccusage` (`ryoppippi/ccusage`, 18,6k★) lit les logs locaux de session et calcule la facture réelle **hors ligne**, sans clé ni compte. Depuis 2026, pas seulement pour Claude Code : Copilot CLI, Codex, OpenCode, Gemini CLI et d'autres sont couverts par le même outil.

© ZENIKA 2026 | All rights reserved

La mémoire de l’agent

Trois familles, du plus explicite au plus automatique, et c’est le seul sujet du chapitre où il n’existe **aucun standard** :

Famille	Exemples 2026	Portée & relecture
Fichier versionné	`AGENTS.md` *(tous les agents)*	équipe, via git · ✓ c’est du code, revu en PR
Mémoire native	**Claude Code** : Auto Memory (par défaut depuis fév. 2026), écrit `MEMORY.md`, plafonné (≈200 lignes/25 Ko), mono-projet · **Copilot Memory** : dépôt (conventions) + préférences utilisateur cross-dépôt · **Codex** : mémoire persistante, opt-in	fonctionnements propriétaires
Middleware tiers	`memoir` *(git-versionné, rappel par chemin)* · `claude-mem` *(désormais « Grok Mem », repositionné xAI)* · serveurs MCP de mémoire	à votre main · ✓ indexée, exportable

© ZENIKA 2026 | All rights reserved

La mémoire : pas un standard

Prescriptif vs descriptif : `AGENTS.md` = ce que l'équipe **décide** ; la mémoire = ce que l'agent a **observé**. Une règle d'équipe n'a rien à faire dans une mémoire auto-générée.

La mémoire : pas un standard. Changer d'agent coûte la **perte de la mémoire**.

© ZENIKA 2026 | All rights reserved

La mémoire de l'agent : la mémoire en action

> **Démarrer simple** : `AGENTS.md` + la mémoire native couvrent 90 % des besoins d'un développeur. `memoir` apporte la puissance quand la mémoire doit être **auditée**, **conservée longtemps** ou partagée **en équipe**. *Ce qui était un plugin tiers en 2025 est devenu, en 2026, à la fois natif chez les assistants et multi-agents chez les outils tiers.*

Ce que capture la mémoire : vos préférences et corrections récurrentes, pour ne plus les répéter d'une session à l'autre.

Session	Ce qui se passe
Session 1	L'agent génère une requête SQL concaténée → vous corrigez *« utilise des requêtes paramétrées »* → l'apprentissage est capturé
Session 2	Sans rappel de votre part, l'agent applique d'emblée des requêtes paramétrées ✓

> La valeur n'est pas de **stocker**, c'est de **ne plus répéter l'erreur**.

© ZENIKA 2026 | All rights reserved

Mémoire persistante : le mécanisme à comprendre

Une mémoire utile ne se **déverse** pas dans la context window : elle s'interroge en **progressive disclosure**, en trois couches.

1



Recherche

Index compact : filtre les IDs pertinents.

COÛT

~50–100 tokens / résultat

2



Chronologie

Contexte chronologique autour des résultats intéressants.

COÛT

contexte ciblé

3



Détail complet

Détails complets, uniquement pour les IDs retenus.

COÛT

~500–1 000 tokens / résultat

i **~10× d'économies de tokens** par rapport à un fetch naïf de toute la mémoire : on paie 50 tokens pour scanner 100 entrées, puis 500–1 000 tokens seulement pour les k résultats filtrés (k << 100).

© ZENIKA 2026 | All rights reserved

Vérifier sa mémoire pour les deux jours

LAB INLINE

Vérifiez que la mémoire est **activée** : vous pourrez relire ce que l'agent a retenu **et ce qu'il a mal retenu** après les **deux jours** de formation.

© ZENIKA 2026 | All rights reserved

Mémoire : ce qu'elle capture

Deux couches à distinguer :

- **Instructions persistantes** : fichiers de règles (`CLAUDE.md`, `AGENTS.md`, etc.).
- **Mémoire évolutive** : faits et préférences apprises au fil des sessions.

Outil	Comment y accéder	Où c'est stocké
Claude Code	`/memory` (ouvrir/éditer) `/context` (vérifier ce qui est chargé)	Instructions : `~/.claude/CLAUDE.md`, `./CLAUDE.md`, `./.claude/CLAUDE.md` Auto memory : `~/.claude/projects/<projet>/memory/`
GitHub Copilot	`/memory` en CLI. On peut aussi interroger l'agent : *« ta mémoire est-elle activée ? où la stockes-tu ? »*	Activée par défaut, en trois portées : utilisateur · dépôt · workspace. Chemins non documentés : c'est l'agent qui vous les donne

À retenir : **Claude expose des fichiers locaux éditables ; Copilot expose une mémoire gérée par la plateforme**, activée sans qu'on l'ait demandé.

© ZENIKA 2026 | All rights reserved

Vérifier sa mémoire en 5 minutes

LAB INLINE 5 min

1. **Localiser** : Claude, `/memory`. Copilot : demandez-lui où il stocke la sienne, puis allez consulter.
2. **Écrire** : donnez-lui un fait à retenir (une préférence de style, un surnom).
3. **Vérifier** : **nouvelle session**, redemandez-le sans le rappeler. S'il répond, la mémoire a traversé la session.

© ZENIKA 2026 | All rights reserved

Réduire les tokens : 5 buffers, 5 leviers

Chaque outil agit sur un **buffer différent** du cycle de l'agent. Pas de recouvrement, donc les gains s'**additionnent**… *à condition que votre usage déclenche réellement chaque levier.*

Buffer ciblé	Levier	Ce qu'il compresse
Sortie shell / outils	rtk (attention)	stdout des commandes (`git` , `ls` , `cat` , `grep` …) : *gain démenti, slide suivant*
Données lourdes	context-mode	gros fichiers, HTTP, logs → sandbox + index
Mémoire inter-sessions	memoir *(ou la mémoire native)*	contexte répété, re-priming
Réponse du modèle	caveman	verbosité, hedging, remplissage
Code généré	ponytail	code écrit maintenant et relu ensuite

Tous compatibles Copilot : nés côté Claude Code, ils se branchent via des **mécanismes standards** (skills, hooks, MCP) et sont devenus multi-agents en 2026.

© ZENIKA 2026 | All rights reserved

Réduire les tokens : le vrai risque

Le vrai risque : empiler des outils qui ne se déclenchent jamais sur *votre* workflow.

Rendre la consommation **visible** (`/context`) la réduit déjà (**effet Hawthorne**).

© ZENIKA 2026 | All rights reserved

Compresser les entrées : rtk & context-mode

rtk : réécrit et compresse la **sortie shell** avant qu'elle n'atteigne le modèle, via un hook `preToolUse` (mécanisme vu plus haut). Mécanisme séduisant… **démenti à la mesure**.

context-mode : exécute les lectures de gros fichiers et les appels HTTP dans un **sandbox**, indexe le résultat et ne renvoie que les **données utiles** (quelques Ko). **17 plateformes**, dont Copilot CLI, VS Code et JetBrains (MCP + hooks).

Outil	Gain annoncé	Gain mesuré *(benchmark JetBrains, juil. 2026)*
rtk	–60 à –90 %	**+7,6 % de coût** (médiane), pendant que son compteur `rtk gain` affichait 96,2 M de tokens « économisés »
context-mode	–98 %	✓ tient : –97,5 % (API GitHub), –99,6 % (logs d'accès)

© ZENIKA 2026 | All rights reserved

rtk : pourquoi le gain ne tient pas

Pourquoi rtk ne tient pas : l'agent lit surtout avec ses **outils natifs**, qui ne passent jamais par le hook shell. Le hook ne voit qu'une fraction du trafic, et le compteur de l'outil mesure une économie **contrefactuelle**, face à une sortie que l'agent n'aurait de toute façon pas produite.

Le réflexe à emporter : un gain auto-mesuré par l'outil lui-même n'est pas une mesure. Le seul chiffre qui compte est celui de **votre** facture, avant / après.

© ZENIKA 2026 | All rights reserved

Compresser les sorties : caveman & ponytail

Deux skills au format standard (donc utilisables sur 30+ agents, Copilot compris) et un enseignement contre-intuitif.

caveman : filtre de style sur la réponse du modèle. Retire articles, remplissage et hedging, garde le code et les erreurs intacts.

ponytail : échelle YAGNI sur le code généré. Stdlib > dépendances, natif > abstraction, suppression > ajout, donc le plus petit code correct.

Outil	Gain annoncé	Gain mesuré *(80 tâches appariées, juil. 2026)*
caveman	−65 %	−8,5 % · gain de coût qui ne survit qu'en excluant un cas atypique
ponytail	~54 % de code	−15,4 % de code · −10,3 % de coût (p = 0,004) · −11 % de temps · qualité stable

© ZENIKA 2026 | All rights reserved

Compresser les sorties : la leçon

La leçon : comprimer la *prose* du modèle rapporte peu (l'entrée domine le coût). Lui faire écrire **moins de code** rapporte vraiment, et améliore aussi la revue et la maintenance.

Mais l'enjeu n'est pas que financier : contraindre un modèle à répondre **court** améliore aussi sa **précision** sur certaines tâches. La brièveté est un levier de qualité avant d'être un levier de coût.

© ZENIKA 2026 | All rights reserved

Pourquoi un RAG pour le code ?

L'agent lit les fichiers à la demande, mais sur un **grand repo**, parcourir la codebase naïvement sature la context window et ralentit chaque requête.

Ce qu'un `AGENTS.md` généré fait déjà : l'agent analyse la structure du projet et produit un `AGENTS.md` avec les conventions, la stack et l'organisation détectées. Une **mémoire statique** suffisante pour les petits projets. Dans VS Code : commande *Chat: Generate Instructions* ; en CLI : il suffit de le demander.

© ZENIKA 2026 | All rights reserved

Quand ce contexte statique ne suffit plus

Besoin	`AGENTS.md` (statique)	RAG (dynamique)
Structure et conventions	✓ Généré une fois	✓ (idem)
« Quelle fonction appelle X ? »	✗ Pas de relations	✓ Graphe de symboles
Repo > 50k lignes	✗ AGENTS.md trop long	✓ Recherche à la demande
Contexte toujours à jour	✗ Refresh manuel	✓ Index watché
Recherche sémantique	✗	✓ Vecteurs

Périmètre de ce chapitre : le RAG **spécifique au code source**, pas le RAG généraliste sur documents. L'objectif est que l'agent navigue intelligemment dans la codebase sans charger des milliers de lignes dans sa context window.

© ZENIKA 2026 | All rights reserved

Trois architectures RAG pour le code

Pour donner à l'agent le **bon contexte sur un grand repo** sans saturer la context window, trois niveaux de sophistication :

1



Chunking naïf

Découpage en blocs de taille fixe (512 tokens) ou par lignes.

> *✓ Simple, rapide à mettre en place · ✓ Bon pour README, docs prose · ✗ Coupe les fonctions en plein milieu · ✗ Perd le contexte syntaxique*

POUR

repos < 10 k lignes · < 5 min

2



Chunking AST-aware

Tree-sitter parse le code et identifie les unités syntaxiques (fonctions, classes, méthodes). 12 langages.

> *✓ Intégrité syntaxique préservée · ✓ +5,5 pts sur RepoEval vs naïf · ✗ Setup 15–30 min*

POUR

10 k–500 k lignes · option intermédiaire

3



Knowledge Graph

Knowledge Graph + AST (tree-sitter, 36 langages, zéro appel API). Skill graphify, au format standard.

> *✓ « Qui appelle cette fonction ? » · ✓ Setup < 5 min · régénérable · ✓ Hook auto-rebuild · rien à démarrer*

POUR

tout type de repo · notre choix

© ZENIKA 2026 | All rights reserved

RAG pour le code : les autres outils du terrain

Autres outils du même terrain, si le besoin dépasse le nôtre :

- **Git Nexus** : interface web interactive, langage de requête, liens interfaces↔méthodes plus fins.
- **Serena** : remplace les recherches `grep` / `glob` de l'agent par une indexation, se branche aussi sur l'IDE.

© ZENIKA 2026 | All rights reserved

Les modes d’accès au RAG

Une fois l’index construit, trois façons de l’exposer à l’agent :

Mode	Fonctionnement	Avantages	Limites
Local direct	Vector DB locale (LanceDB, ChromaDB) · requêtes via SDK Python · latence ~ms	Données privées · contrôle total · coût zéro	Setup individuel · non partageable facilement
Skill + CLI ✓	Le `SKILL.md` décrit les commandes à l’agent, qui les lance lui-même (`query` , `path` , `explain`)	Rien à démarrer · format portable · versionnable dans le dépôt	Dépend du skill fourni par l’outil
Cloud intégré	Cursor Index · Windsurf Cascade · Augment Code : indexation auto, zéro config	Zéro friction · toujours à jour	Données envoyées hors site · moins configurable

© ZENIKA 2026 | All rights reserved

Outil recommandé en 2026 : Graphify

```
# installation
uv tool install graphifyy
# pose le SKILL.md (ou : claude, codex…)
graphify install --platform copilot
# construit le graphe (graphify-out/)
graphify .
# auto-rebuild sur commit / checkout
graphify hook install
```



Recommandation : **Graphify**. Skill au format standard, **rien à démarrer**, et un hook qui **maintient le graphe à jour** au lieu de le laisser dériver.

© ZENIKA 2026 | All rights reserved

Quel RAG pour quel projet ?

Critère	Chunking naïf	AST-aware	Knowledge Graph (skill)
Taille du repo	< 10 k lignes	10 k – 500 k lignes	Tout type de repo
Requêtes de relations	✗	Limitées	✓
Données locales	✓	✓	✓
Partageable en équipe	Non (index local)	Config versionnée	Chacun régénère · hook git ✓
Exemples d'outils	LlamaIndex · LangChain	cocoindex · Continue.dev	Graphify · Git Nexus · Serena



Sur un **vrai projet d'équipe** : `graphify hook install`, et chacun a un graphe à jour sans y penser.
Inutile de commiter `graphify-out/` — c'est un artefact dérivé, régénérable en quelques secondes.

© ZENIKA 2026 | All rights reserved

Lab 1.3 · Mémoire, tokens et indexation

Indexer sa codebase

Objectif : Graphify installé, knowledge graph de la PetClinic construit, requêtes sémantiques fonctionnelles.

Consignes : handout « Lab 1 · Boîte à outils agents personnels », étape 1.3.



© ZENIKA 2026 | All rights reserved

Ce qu’on retient du chapitre 2

Maîtriser les tokens : `/context` d’abord (on n’optimise pas ce qu’on ne mesure pas), puis compaction, caching, agents bornés, choix du modèle. La dépense, elle, se lit ailleurs (`ccusage` , `/cost` , tableau de bord éditeur) : deux questions distinctes, deux unités distinctes.

Réduire les tokens, avec un esprit critique : les outils de compression de sortie (caveman, ponytail) rapportent peu ; c’est faire écrire **moins de code** à l’agent, ou rendre la consommation visible, qui rapporte vraiment. Un gain auto-mesuré par l’outil lui-même n’est pas une mesure.

Mémoire : trois familles (fichier versionné · native · middleware tiers) et le **seul artefact du chapitre sans standard**, donc le seul qui ne se transfère pas quand on change d’agent. `memoir` (git-versionné) est la recommandation actuelle côté middleware ; `claude-mem` /Grok Mem a ouvert la voie mais s’est recentré commercialement ailleurs.

RAG sur le code : Graphify combine AST tree-sitter (36 langages) + knowledge graph. Skill au format standard + hook git d’auto-rebuild = contexte à jour sans saturer la context window. Alternatives à connaître : GitNexus, Serena, JetBrains Context.

© ZENIKA 2026 | All rights reserved

Chapitre 2 : le fil rouge

Le hook `preToolUse` du LAB 2/2 du **Ch. 1** revient ici pousser l'agent vers le graphe Graphify : la boîte à outils personnelle du développeur est **bouclée**.

Suite au Ch. 3 : la spécification comme méthode d'équipe.

© ZENIKA 2026 | All rights reserved

Chapitre 3

Spec-Driven Development : fondations

User stories, frameworks SDD, et un premier Pomodoro spécifié avant d'être codé.



© ZENIKA 2026 | All rights reserved

Peut-on vraiment générer des specs ?

L'essentiel du travail d'un Product Owner (PO), c'est de **discuter le besoin** avec les parties prenantes.

Les assistants IA sont là pour :

- **formaliser** les user stories et la documentation,
- repérer les **dépendances** possibles dans le code.

> *i* Ils **ne remplacent pas** les conversations avec les utilisateurs.

© ZENIKA 2026 | All rights reserved

AI-Driven Dev vs Spec-Driven Dev

La même démarche, mais ancrée sur un artefact différent :

- AI-Driven Dev : `architecture-backend.md` (instructions) → **prompt / plan d'exécution** → code
- Spec-Driven Dev : `product-mgmt-guidelines.md` (instructions) → **description de user story** → code

> *i* On déplace la **source de vérité** du prompt ponctuel vers une **spécification** durable.

© ZENIKA 2026 | All rights reserved

Générer des user stories : avec ou sans guidelines

Générer une user story, c'est comme générer du code : **sans guidelines, on n'obtient pas ce qu'on veut**.

Sans guidelines, la story générée est typiquement :

- trop **verbeuse**,
- pas dans **notre syntaxe** préférée,
- trop **technique**,
- **sans Definition of Done**.

> *i* Il faut donc des **instructions** dédiées à l'écriture des stories.

© ZENIKA 2026 | All rights reserved

Mettre en place les instructions

Un exemple de `story-writing-guide.md` :

```
### Story Template
Une story suit ce gabarit :
{{User Story}}
{{Acceptance Criteria}}
{{Screenshots}}
{{Definition of Done}}
{{Notes}}

### User Story
Format : « As a [userType], I want [action] so that [businessValue]. »
```

> *i* Ce n'est qu'un **exemple** : chaque PO a sa meilleure façon d'écrire ses stories.

© ZENIKA 2026 | All rights reserved

Lancer la génération de user stories

- Indiquer le **dossier de guides** à lire dans le prompt.
- Travailler en mode **Accept Edits**.
- **N'importe quel grand modèle** convient : générer une story demande **moins de puissance** que générer du code.



La user story générée est un **livrable** : on la relit, on la corrige, on la range (ex. `docs/stories`).

© ZENIKA 2026 | All rights reserved

Lab 2.1 · 2.2 · User Stories

Votre définition d’une bonne spec, puis une user story



En deux temps, sur la PetClinic : définir votre bonne spec, puis utiliser votre guide.

Vous redevenez garants de ce que « bonne spec » veut dire, au lieu de copier-coller un prompt.

Consignes : handout « Lab 2 · User stories », étapes 2.1 et 2.2.

© ZENIKA 2026 | All rights reserved

Lab 2.1 · 2.2 : les deux temps

LAB 2

1. **Définir** : brainstormer *votre* définition d'une bonne spec, puis l'**encoder en skill** (`.github/skills/story-writing/`). *(Concentrez-vous sur le **fond** plus que sur la forme.)*
2. **Utiliser** : générer une user story en suivant **le guide que vous venez d'écrire**.

> *i* Une story cadrée par *votre* guide est **concise, dans votre syntaxe, avec une Definition of Done.**

© ZENIKA 2026 | All rights reserved

Les frameworks de Spec-Driven Development

Il existe de nombreux frameworks. Trois reviennent souvent :

- **OpenSpec** : https://github.com/Fission-AI/OpenSpec
- **Spec Kit (GitHub)** : https://github.com/github/spec-kit
- **BMAD** : https://github.com/bmad-code-org/BMAD-METHOD

> *Spec-Driven Dev with… you name it.* L'important n'est pas le framework, mais la **spec comme source de vérité**.

© ZENIKA 2026 | All rights reserved

OpenSpec

- Framework **léger** pour le Spec-Driven Development.
- **Simple et facile** : une bonne porte d'entrée.
- A atteint la **version 1.0 en janvier 2026**.
- Communauté active (Discord).

© ZENIKA 2026 | All rights reserved

> *i* Idéal pour démarrer et **ressentir** la démarche avant de passer à des frameworks plus riches.

Mettre en place OpenSpec

```
# Installer OpenSpec (paquet npm global)
npm install -g @fission-ai/openspec@latest

# Initialiser au niveau du projet
cd mon-projet
# sélectionner l'agent (Copilot…) ; ajoute prompts et skills au projet
openspec init
```

Puis, dans le chat (après rechargement de l'éditeur) :

```
/opsx-propose
Je veux construire une application Pomodoro simple dans le dossier
Pomodoro (HTML et JavaScript, côté client uniquement).
```

> *i* `opsx` = raccourci d'« OpenSpec ». Workflow : **propose → apply/test → archive**.

© ZENIKA 2026 | All rights reserved

Exemple de specs générées

```
openspec/
├── specs/                  # Source de vérité (comportement du système)
│   └── pomodoro/
│       └── spec.md
├── changes/                # Changements proposés (un dossier par change)
│   └── create-pomodoro-app/
│       ├── proposal.md
│       ├── design.md
│       ├── tasks.md
│       └── specs/          # Delta specs (ce qui change)
│           └── create-page/
│               └── spec.md
└── config.yaml             # Configuration du projet (optionnel)
```



Les **specs** sont versionnées : on régénère / fait évoluer le projet à partir d'elles. L'étape **archive** d'openspec synchronise les specs avec ce qui est vraiment implementé après l'apply

© ZENIKA 2026 | All rights reserved

GitHub Spec Kit

Framework de dev agile piloté par l'IA, par **GitHub** (Den Delimarsky).

```
# Paquet Python, via uv
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
# 30+ agents supportés ; ici Copilot
specify init <PROJET> --integration copilot
```

> *i* Plus d'étapes qu'OpenSpec, mais **le plus adaptable** : extensions, presets et bundles (Ch. 4). Installé à la racine du workspace.

© ZENIKA 2026 | All rights reserved

Spec Kit : le workflow par commandes

Commande	Produit
`/speckit.constitution`	`constitution.md`
`/speckit.specify`	`spec.md` (user stories), checklists
`/speckit.plan`	`plan.md`, `data-model.md` …
`/speckit.tasks`	`tasks.md`
`/speckit.implement`	produit le code applicatif, et check les tasks

© ZENIKA 2026 | All rights reserved

BMAD

« Breakthrough Method of agile AI-driven dev » (Brian Madison). Communauté très active.

- 20+ agents spécialisés (analyst, architect, dev, pm…) et des workflows guidés.
- Adresse le greenfield ET le brownfield.

```
npx bmad-method@alpha install   # choisir l'agent (Copilot…) + les modules
```

Quick Flow	Standard Flow
Mise en place rapide (petit changement)	≥ 30 min de mise en place
Un seul agent (solo-dev)	Plusieurs agents (analyst, architect, dev, pm…)
Travail en Tasks + critères d'acceptation	Travail en Epics & User Stories

© ZENIKA 2026 | All rights reserved

Notre recommandation

- Le SDD va **s'étendre en 2026**. Certains parlent de **« l'année du Spec-Driven Development »**.
- **Expérimentez** OpenSpec, BMAD et Spec Kit :
  - sur des **pet projects**,
  - sur des projets **petits et non critiques**,
  - en gardant un œil sur leur évolution **rapide**.

> *i* L'angle directeur : viser une **documentation réellement à jour** (fonctionnelle + technique), qui permet de **repartir sur des bases maîtrisées**.

> *i* **Règle minimale, quelle que soit la maturité de l'équipe** : quel que soit le framework choisi (ou l'absence de framework), la **spécification précède la génération de code**. C'est le seuil minimal du SDD, pas une option parmi d'autres.

© ZENIKA 2026 | All rights reserved

Lab 3.1 · Pomodoro (1/2)

Construire un Pomodoro avec OpenSpec

Objectif : spécifier puis générer une petite app Pomodoro (HTML + JS, côté client) avec OpenSpec.

On a fait spec d'abord, code ensuite, et la spec reste la source de vérité régénérable.

Consignes : handout « Lab 3 · Pomodoro », étape 3.1.



© ZENIKA 2026 | All rights reserved

Lab 3.2 · Pomodoro (2/2)

Construire le Pomodoro avec Spec Kit



Objectif : refaire le Pomodoro avec GitHub Spec Kit, en gardant la charte visuelle de l'étape précédente et en parcourant tout le cycle Spec Kit.

À vous de jouer : plusieurs chemins fonctionnent.

Consignes : handout « Lab 3 · Pomodoro », étape 3.2.

© ZENIKA 2026 | All rights reserved

Chapitre 4

Workflow adapté à l'IA

Postures face à la spec, extensions et presets, garde-fous de commit.



© ZENIKA 2026 | All rights reserved

Trois postures face à la spec

Avant de personnaliser un framework, il faut savoir **où l'on veut aller**. Birgitta Böckeler (*SDD: the 3 tools*, martinfowler.com) distingue **trois postures** qui jalonnent l'adoption :

1



Spec-first

La spécification est écrite d'abord et sert de prompt structuré. Mais elle n'est pas maintenue : elle peut diverger du code édité ensuite à la main.

2



Spec-anchored

Spec et code synchronisés en permanence. Le code reste éditable, mais chaque évolution est ancrée dans une spec à jour ; les divergences sont détectées.

3



Spec-as-source

Seule la spec est éditée. Le code devient un artefact généré, comme le binaire d'un compilateur : modifier la spec régénère le système.

© ZENIKA 2026 | All rights reserved

Aucun framework ne fait (encore) du spec-as-source

Parmi les trois frameworks vus au chapitre 3 (BMAD, Spec Kit et OpenSpec), **aucun n'atteint le *spec-as-source*** :

- ils restent au mieux **spec-first / spec-anchored** ;
- les **features et spécifications** sont **difficilement maintenues dans la durée** : elles divergent du code dès qu'on édite à la main.

> *i* Seul le **rejeu incrémental du dépôt git** (rejouer toute la suite de changements de specs) pourrait reconstruire un résultat équivalent… **non envisageable sur le long terme** (coût, dérive, non-déterminisme).

© ZENIKA 2026 | All rights reserved

Spec-centric ou workflow-centric : une autre façon de trancher

Les trois frameworks du Ch. 3 parient tous sur un **artefact durable** : la spec. Un autre pari existe, tout aussi sérieux : parier sur la **discipline d'exécution** plutôt que sur l'artefact.

Superpowers (`obra/superpowers`, 287k★) l'incarne : pas de spec centrale, mais une chaîne de *skills* qui impose un **process** : brainstorm → worktree isolé → plan d'implémentation → exécution par sous-agent avec **double revue** → TDD rouge-vert-refactor → merge. Fonctionne sur Claude Code, Cursor, Codex, Copilot CLI, Gemini CLI, OpenCode.

© ZENIKA 2026 | All rights reserved

Spec-centric ou workflow-centric : la comparaison

	Spec-centric (BMAD · Spec Kit · OpenSpec)	Workflow-centric (Superpowers)
Artefact de référence	la spec, versionnée	la discipline, encodée en skills
Ce qui survit à la session	un document relisible	un process reproductible
Point fort	la spec sert de mémoire longue au projet	boucle d'implémentation très serrée

Pas un concurrent à départager : Spec Kit quand la spec doit survivre à la session et être lue par des non-agents ; Superpowers pour la discipline d'exécution serrée sur une tâche bornée. Les deux se **combinent** plutôt qu'ils ne s'excluent.

© ZENIKA 2026 | All rights reserved

Ce que l'IA change dans notre façon de travailler

Le besoin n'est plus « écrire du code plus vite » : c'est de **piloter tout l'enchaînement, de la spécification jusqu'aux tests, dans un temps très réduit**. Or nos garanties reposent sur des **rendez-vous humains** (revue, point d'équipe, recette), calibrés sur un rythme de production qui n'existe plus.

	Hier	Avec un agent
Rythme	une feature = des jours	une feature = des heures
Ce qui garantit	revues et rituels, *a posteriori*	contrôles systématiques, à chaque étape

i Les revues et les rituels **ne disparaissent pas** : ils ne suffisent plus **à eux seuls**. On leur adjoint des contrôles **automatiques et systématiques**, qui tiennent le rythme de l'agent. **Le SDD est *une* réponse possible** : une source de vérité durable, et des étapes où brancher ces contrôles.

© ZENIKA 2026 | All rights reserved

Le sujet qui dépasse l’outillage

Ce changement a des **impacts forts sur l’organisation de l’équipe** : qui écrit la spec, qui la valide, ce que devient la revue, comment on estime et on découpe.

Ces impacts sont spécifiques à chaque équipe : il n’existe pas de modèle à copier, et cela ne se décide pas en formation.

© ZENIKA 2026 | All rights reserved

Des fonctionnalités encore à venir

Le sujet est **identifié** et **ouvert** côté frameworks, la maintenance de la spec dans le temps est l'angle mort à combler :

Framework	Piste / ticket
Spec Kit	*Evolving specs as features are added*, issue #916 ; *Update/refine existing specs*, issue #1191
OpenSpec	*Spec reuse : auto-load archived specs*, issue #802
BMAD	roadmap publique ; *PRD lifecycle after MVP*, discussion #1838

i **Conviction Zenika** : ces fonctionnalités **pourraient** arriver (rien n'est acquis). L'écosystème reste aujourd'hui orienté **code-as-source** : le code fait référence… ce qui, **par expérience, ne passe pas l'épreuve du temps**. Le **code n'est pas le bon niveau d'abstraction**, même pour une IA, *a fortiori* avec des **context windows limitées**. D'où le cap vers le **spec-as-source**.

© ZENIKA 2026 | All rights reserved

D’où la nécessité de customiser

Deux raisons convergent :

1. **Combler le manque** : pousser ces frameworks vers le *spec-anchored* / *spec-as-source* plutôt que d’attendre.
2. **Coller à l’entreprise** : optimiser le workflow pour **nos pratiques** (revue, Jira, presets, phase de design…).

> *i* Il faut donc pouvoir **customiser les workflows** proposés par ces frameworks.

© ZENIKA 2026 | All rights reserved

La spec n’est qu’une partie de la doc

La spec décrit le **QUOI**. Mais ce qu’on doit **conserver dans le temps**, c’est la **documentation**, *fonctionnelle **et** technique*, tenue à jour et **régénérable**.

Documentation : le patrimoine à conserver, à jour, régénérable, lisible par l’humain et l’IA :



Spec fonctionnelle

le QUOI · l’intention

CONTIENT

user stories · critères



Doc technique

archi · décisions

CONTIENT

invariants · runbooks



Code : artefact généré

la documentation le régénère

i **Conviction Zenika** : le **code** est un artefact, la **spec** une **partie** de la doc ; c’est la **documentation complète** qui est le patrimoine. Customiser le workflow doit donc **produire et maintenir cette doc**, pas seulement la spec.

© ZENIKA 2026 | All rights reserved

Extensions : ajouter des capacités

Une **extension** apporte de **nouvelles capacités** au-delà du cœur : nouvelles **commandes**, **templates**, intégrations d'outils, **quality gates**, voire une **nouvelle phase** de workflow.

```
specify extension search [requête]      # découvrir le catalogue
specify extension add <nom>             # installer (--from <url>, --dev, --priority N)
specify extension list                  # lister les extensions installées
specify extension info <nom>            # détails + commandes apportées
specify extension remove <nom>          # désinstaller
```

Requiert un projet déjà initialisé (`specify init`). Une commande ajoutée apparaît sous la forme `/speckit.ext.<nom>` .

Mini-lab : `specify extension search doc` → repérez les extensions qui traitent de la documentation (génération, synchronisation, audit de couverture).

© ZENIKA 2026 | All rights reserved

Presets : changer le comportement

Un **preset** change **comment** Spec Kit travaille **sans ajouter de capacité** : il **surcharge** les templates, commandes et la terminologie du cœur (ex. format de spec **conforme**, **standards d'entreprise**).

```
specify preset search [requête]          # découvrir
specify preset add <preset>              # installer (catalogue ou URL)
specify preset list                      # lister
specify preset info <preset>             # détails
specify preset remove <preset>           # désinstaller
```

Résolution par **priorité** : *project-local* > **presets** > **extensions** > cœur Spec Kit. **Extension = nouvelles capacités ; preset = surcharge des existantes.**

© ZENIKA 2026 | All rights reserved

Bundles : composer extensions et presets

Nouveau : les bundles (`specify bundle` , depuis la **0.11.4**, juin 2026). Un bundle **compose** extensions + presets + workflows en **une unité versionnée** : *« tout ce dont une équipe ou un rôle a besoin »*, installé en une commande. Aucun comportement nouveau : c'est une couche de **distribution**.

Mini-lab : `specify preset search` → le catalogue est encore court. `pirate` en est l'exemple canonique : il change le vocabulaire et le ton de Spec Kit.

© ZENIKA 2026 | All rights reserved

Et chez BMAD ? Customiser sans forker

Même but, autre mécanique : **surcharges TOML** dans `_bmad/custom/`, on **n'édite jamais** les fichiers installés, et les overrides **survivent aux mises à jour**.

- **Trois couches** : `{workflow}.user.toml` (perso) > `{workflow}.toml` (équipe) > `customize.toml` (défauts).
- On peut **injecter du contexte** (`persistent_facts`, refs `file:`), **ajouter des étapes** (`activation_steps_prepend/append`), **changer la sortie** (`on_complete`).
- Skill guidée `bmad-customize` : repère le customisable, écrit l'override, vérifie le merge.
- Plus loin (nouveaux **agents / phases / domaines**) : **expansion packs** (`expansion-packs/`) + **BMad Builder**.

```toml
# _bmad/custom/bmad-product-brief.toml  (équipe, versionné)
[workflow]
activation_steps_prepend = ["Charger docs/product/principles.md."]
persistent_facts = ["file:{project-root}/_bmad/custom/guidelines.md"]
on_complete = "Résumer en bullets + proposer un envoi."
```

> *i* **Spec Kit** = extensions + presets · **BMAD** = overrides TOML + expansion packs. Même principe : **personnaliser sans forker**.

© ZENIKA 2026 | All rights reserved

Et chez OpenSpec ? Config projet et schemas

OpenSpec se personnalise aussi, sur **trois niveaux** : de la simple config aux workflows maison.

Niveau	Rôle	Pour qui
Config projet (`openspec/config.yaml`)	Schéma par défaut, contexte et règles injectés	La plupart des équipes
Schemas custom (`openspec/schemas/`)	Ses propres artefacts et dépendances	Process spécifique
Surcharge globale (`~/.local/share/openspec/schemas/`)	Schemas partagés entre projets	Utilisateurs avancés

© ZENIKA 2026 | All rights reserved

OpenSpec : la config projet

```yaml
# openspec/config.yaml  (versionné avec le code)
schema: spec-driven

context: |
  Stack : TypeScript, React, Node.js, PostgreSQL
  Tests : Jest + React Testing Library

rules:            # uniquement pour l'artefact visé
  proposal:
    - Inclure un plan de rollback
  specs:
    - Utiliser le format Given/When/Then

operations:       # conseils pour apply et archive
  apply:
    guidance:
      - Lancer les tests ciblés avant la suite complète
```

i Le **contexte** est injecté dans **tous** les artefacts ; les **règles** seulement dans l'artefact correspondant.

© ZENIKA 2026 | All rights reserved

Schemas : définir son propre workflow

Un **schema** décrit les **artefacts** du workflow et leurs **dépendances** (`requires`). On part d'un schema existant ou de zéro.

```
openspec schema fork spec-driven with-review   # forker le schema par défaut
openspec schema init rapid --artifacts "proposal,tasks"   # ou partir de zéro
openspec schema validate with-review           # syntaxe, templates, cycles
openspec schema which --all                    # d'où vient chaque schema ?
```



Mini-lab : forkez `spec-driven`, ajoutez un artefact `review`, validez avec `openspec schema validate`, puis `openspec new change demo --schema with-review`.

© ZENIKA 2026 | All rights reserved

Schemas : ajouter une phase de revue

```yaml
# openspec/schemas/with-review/schema.yaml  (extrait)
artifacts:
  - id: review
    generates: review.md
    template: review.md          # templates/review.md
    instruction: |
      Créer une checklist de revue : sécurité, performance, tests.
    requires: [design]

  - id: tasks
    generates: tasks.md
    requires: [specs, design, review]   # les tâches exigent la revue
```

Nouvelle phase = nouvel artefact (ex. `design`, `review`, contrat d'API). Les **templates** guident le contenu, `instruction` guide l'IA. Des **schemas communautaires** existent (TDD + revue adversariale, ML, E2E…) : on copie le dossier dans `openspec/schemas/`.

© ZENIKA 2026 | All rights reserved

Un gradient de personnalisation

Customiser un workflow se fait par **étapes**, selon la maturité de l'équipe :

1. **Framework rigide** (clé en main) : on adopte tel quel.
2. **À la main** : on s'inspire de l'existant, on ajuste au cas par cas.
3. **Framework personnalisé entreprise** : un workflow maison *opinionated*, sur un **vrai projet**.

> *i* « Tordre » un framework n'est **pas toujours simple**, surtout sur un **projet existant**, d'où l'intérêt de partir des **presets / extensions** vus juste avant.

© ZENIKA 2026 | All rights reserved

Une skill pour (re)générer la doc

Premier réflexe d'un workflow maison : une **skill déclenchable** qui **génère ou met à jour** la **documentation complète** du projet en cours.

- **Déclenchement** : *« mets à jour la doc »* ou une commande dédiée (`/doc`).
- **Entrées** : le **code**, les **specs**, la doc existante → elle calcule les **deltas** et signale les **écarts**.
- **Sorties** (versionnées) :

```
docs/
├── overview.md          # vision produit + périmètre
├── architecture.md      # composants, flux, dépendances
├── decisions/           # ADR (décisions + justifications)
├── modules/<name>.md    # règles métier, invariants
└── runbooks/            # exploitation, incidents
```

> *i* La doc devient un **livrable régénérable à la demande**, plus une corvée qu'on repousse.

© ZENIKA 2026 | All rights reserved

Garde-fous : du code au commit

En SDD, l'IA génère **vite** : sans garde-fou, le **drift** est invisible. On en cumule **trois** :

- **Tests** : le contrat exécutable : unit, intégration, et désormais E2E (voir Ch. 5). *Le score de mutation, pas seulement la couverture, complète le tableau (cf. slide « Deux natures de tests »).*
- **Revue agentique** : analyse avant d'envoyer, **par un agent distinct de celui qui a généré le code**. En local aujourd'hui, **en CI au Module 3**.
- **Le commit lui-même** : le moment naturel pour vérifier que **la doc suit** et que **la part d'IA est tracée**.

> *i* Le commit n'est pas qu'un historique : c'est un **point de synchronisation** entre le code, la documentation et la traçabilité de l'outillage IA utilisé. C'est l'objet du lab qui suit.

> *i* **Règle minimale, sans exception** : aucun code généré par IA ne passe en production sans avoir franchi une **vérification déterministe** : tests, analyse statique, **revue agentique distincte de l'agent générateur**.

© ZENIKA 2026 | All rights reserved

SPEC.md : un format inspiré d'OpenAI Symphony

`SPEC.md` est inspiré du `SPEC.md` du projet Symphony (OpenAI) : un document unique, agnostique au langage, avec **huit sections normatives** (§1–§8) et un vocabulaire RFC 2119 (`MUST` / `SHOULD` / `MAY`).

```
SPEC.md
├── Status / Purpose              ← en-tête
├── §1  Problem Statement
├── §2  Goals & Non-Goals
├── §3  System Overview           ← technique
├── §4  Domain Model
├── §5  Functional Behavior       ← fonctionnel
├── §6  Technical Architecture
├── §7  Glossary
├── §8  References
├── ## Implementation Status      ← opérationnel
└── ## Pending SPEC Updates       ← drift détecté
```

Pas de standard industriel : ce format est une **proposition** d'OpenAI, pas un consensus. Il est ici parce qu'il répond bien au besoin d'un agent qui doit lire *toute* la doc d'un coup.

© ZENIKA 2026 | All rights reserved

Le commit comme porte d'entrée

Deux informations **systématiquement absentes** des commits IA aujourd'hui :

Traçabilité IA

Quelle part du code est générée par l'IA ? Aucun standard aujourd'hui.

```
feat(auth): add refresh token

AI-Generated: ~80%
```

→ footer de commit standardisé

SPEC.md à jour ?

Le code a changé : la spec aussi ? Personne ne vérifie avant de pusher.

L'extension détecte un drift entre le diff stagé et `SPEC.md`, et bloque si une mise à jour est attendue.

→ quality gate avant le commit

Solution : une **extension Spec Kit** (`speckit-doc-tracker`) qui intercale ces deux vérifications dans la commande de commit, *avant* que le message ne soit finalisé.

© ZENIKA 2026 | All rights reserved

On assume : co-signer le travail fait avec l'IA

Le principe de base : on assume.

Le travail est fait avec de l'IA, on le **co-signe** (`Co-Authored-By:`) et on garde la **proportion IA / humain**.

Effacer la trace, c'est se priver de la seule mesure dont on disposera plus tard.

© ZENIKA 2026 | All rights reserved

Architecture du besoin : quel mécanisme choisir ? (1/2)

Plusieurs façons d'automatiser la traçabilité IA et la vérification de SPEC.md au commit :

Approche	Fonctionnement	Avantages / Limites
Preset qui surcharge toutes les commandes	Injecte le contrôle doc à chaque `/specify`, `/plan`, `/implement`…	Exhaustif · Intrusif, lourd en contexte
Skill dédiée au commit	Une commande `/commit-assistant` à appeler manuellement	Léger, explicite · Il faut penser à l'invoquer
Extension embarquée dans le projet	Ajoute `/speckit.doc-tracker.commit`, versionnée avec le code	Simple, portable · Un seul point d'entrée ← **notre choix**

© ZENIKA 2026 | All rights reserved

Architecture du besoin : quel mécanisme choisir ? (2/2)

Approche	Fonctionnement	Avantages / Limites
Preset + extension	Le preset détecte quand la doc doit être vérifiée et déclenche l'extension	Probablement idéal · Plus long à mettre en place
Schema OpenSpec	Un artefact dédié dans un schema custom (`openspec/schemas/`)	Déclaratif, versionné · Pas de commande de commit ni de contrôle bloquant
Feature native du framework	Intégrée directement dans Spec Kit / BMAD / OpenSpec	La bonne réponse à terme · Pas encore disponible



La feature native est **en débat dans les communautés** des frameworks SDD : cf. les issues évoquées plus tôt dans ce chapitre (slide « Des fonctionnalités encore à venir »).

© ZENIKA 2026 | All rights reserved

Lab 4.1 · Extension speckit-doc-tracker (1/3)

Créer et installer l'extension

Objectif : créer une extension Spec Kit `speckit-doc-tracker` qui enrichit chaque commit d'un indicateur de part de code généré par l'IA et d'une vérification de SPEC.md.

Committer via l'assistant de code (pas en ligne de commande directe) rend la traçabilité possible : il connaît le contexte complet du diff.

Consignes : handout « Lab 4 · Extension speckit-doc-tracker », étape 4.1.



© ZENIKA 2026 | All rights reserved

Lab 4.2 · Extension speckit-doc-tracker (2/3)

La commande de commit enrichie

L'estimation de la part IA est approximative par nature : ce qui compte, c'est d'établir une convention d'équipe cohérente et de la rendre visible dans l'historique git.

Consignes : handout « Lab 4 · Extension speckit-doc-tracker », étape 4.2.



© ZENIKA 2026 | All rights reserved

Lab 4.3 · Extension speckit-doc-tracker (3/3)

SPEC.md à jour avant chaque commit

Le commit devient un point de synchronisation entre le code et la spec : ni contraignant, ni silencieux.

Consignes : handout « Lab 4 · Extension speckit-doc-tracker », étape 4.3.



© ZENIKA 2026 | All rights reserved

Context Nexus : la doc comme infrastructure agentique

Zenika Context Nexus organise *toute* la connaissance de l'organisation :

Registre	Ce que c'est	Artefacts concrets	Délégable à un agent ?
Intent	Ce que l'organisation décide : standards, contraintes non négociables	`AGENTS.md` · `.github/instructions/*.md` · constitution Spec Kit · hook `preToolUse`	✗ Jamais
Knowledge	Ce que l'organisation sait : décisions passées et leur contexte, incidents	ADR (`docs/decisions/`) · post-mortems · liens vers la documentation d'entreprise	✓ Rédaction assistée, ratification humaine
Contracts	Ce que l'organisation garantit : seuils mesurables, DoD, SLO	Critères d'acceptation · `docs/contracts/` · quality gates de `doc-tracker`	✗ Jamais
Operations	Comment l'organisation travaille : procédures, patterns récurrents	Skills · agents custom · hooks · templates `.specify/`	✓ Oui

© ZENIKA 2026 | All rights reserved

Context Nexus : l’ambition

POUR ALLER PLUS LOIN

> *« La pertinence d’un agent est plafonnée par la qualité du contexte qu’on lui fournit. »*

L’ambition est plus large qu’une spécification.

La spec décrit *un produit*. Le Context Nexus décrit *une organisation qui produit*, et le rend exploitable par des agents sans réécriture.

© ZENIKA 2026 | All rights reserved

Deux natures de tests, deux publics

Quand l'IA génère les tests aussi, le piège est d'en obtenir **beaucoup**, et d'hériter d'une suite illisible, à retoucher au moindre changement.

	Tests de construction	Tests fonctionnels
À qui ils servent	à **l'agent**, pour s'autocontrôler pendant qu'il code	à **l'équipe**, pour savoir ce que fait le produit
Ce qu'ils testent	l'implémentation, unité par unité	les **effets de bord** observables, indépendamment du comment
Durée de vie	jetables : ils accompagnent la génération	durables : ils *sont* la spec exécutable
On les relit ?	non	oui, ligne à ligne

> **La règle de tri, en fin de feature** : là où les deux se recouvrent, **c'est le test fonctionnel qui gagne**. On supprime le doublon de construction plutôt que de maintenir deux fois la même vérité.

© ZENIKA 2026 | All rights reserved

Tests générés : le symptôme et le score de mutation

Le symptôme à reconnaître : une suite où toucher la moindre ligne oblige à retoucher les tests (mocks partout, une classe = un test). Ce n'est pas de la rigueur, c'est du couplage.

Un angle complémentaire, pour les deux natures : le **score de mutation** (Stryker, PIT, mutmut, cargo-mutants selon l'écosystème) modifie mécaniquement le code (`<` → `⩽` , supprime une condition…) et vérifie que la suite **échoue**. Une IA qui génère code **et** tests peut faire monter la couverture sans vérifier grand-chose ; le score de mutation est bien plus dur à tricher, parce qu'il mesure si les tests **détectent** un vrai changement de comportement, pas seulement s'ils s'exécutent.

© ZENIKA 2026 | All rights reserved

Ce qu’on retient du chapitre 4 (1/2)

Trois postures, un curseur de maturité : spec-first → spec-anchored → spec-as-source. Aucun framework n’y est encore, mais on peut s’en approcher.

Customiser sans forker : extensions (nouvelles capacités) et presets (surcharges) pour Spec Kit ; overrides TOML pour BMAD. La personnalisation survit aux mises à jour.

Un seul fichier de vérité : `SPEC.md` (inspiré d’OpenAI Symphony) centralise spec fonctionnelle et technique : format de référence, pas encore un standard industriel.

© ZENIKA 2026 | All rights reserved

Ce qu'on retient du chapitre 4 (2/2)

Le commit comme garde-fou : l'extension `speckit-doc-tracker` trace la part d'IA et vérifie SPEC.md avant chaque commit. Committer via l'assistant rend ces contrôles possibles.

Context Nexus : vision plus ambitieuse, quatre registres (Intent, Knowledge, Contracts, Operations) pour que la documentation devienne une **infrastructure agentique**. Ce qu'on a fait ici est déjà l'Étape 1.

> **Fil rouge** : l'extension `speckit-doc-tracker` et les quatre registres seront mis en pratique dans le **TP E2E du Ch. 5** : documentation et logs vérifiés commit par commit.

© ZENIKA 2026 | All rights reserved

Le « shift-up » : la conception redevient un geste de développeur

Le *shift-left* déplaçait des activités de test **vers l'amont**, mais restait sur le même terrain (le code). Un mouvement plus récent, désigné dans la recherche 2025-2026 sous le nom de **« shift-up »**, déplace autre chose : puisque l'implémentation de détail se délègue à l'agent, le développeur remonte vers la **spécification et la conception d'architecture** — pas comme spectateur d'un artefact produit et remis par le produit/l'UX, mais comme **auteur**.

> *i* C'est exactement le geste que fait le lab qui suit : le wireframe (ou le contrat d'API) n'est pas dessiné par un designer puis remis à l'équipe — il est écrit par le développeur, **avant** que l'agent ne génère quoi que ce soit.

© ZENIKA 2026 | All rights reserved

Lab 4.4 · facultatif · Spec Kit perso (1/3)

Ajouter une phase de design

Lab facultatif : à parcourir à l'oral comme illustration complémentaire, pas à réaliser en autonomie.

Objectif : étendre Spec Kit pour intercaler un wireframe basse fidélité (HTML) entre `specify` et `plan`. On insère une étape maison dans le pipeline : un deuxième exemple d'extension après `doc-tracker`.

Consignes : handout « Lab 4.4 à 4.6 · Phase de design (facultatif) », étape 4.4.



© ZENIKA 2026 | All rights reserved

Lab 4.6 · facultatif · Spec Kit perso (2/3)

Variante backend : une phase de contrat d'API



Lab facultatif : même mécanique, terrain différent. L'un ou l'autre selon le groupe.

Objectif : intercaler la génération d'un contrat OpenAPI entre `specify` et `plan` : le contrat fixe chemins, schémas et codes retour *avant* l'implémentation.

Consignes : handout « Lab 4.4 à 4.6 · Phase de design (facultatif) », étape 4.6.

© ZENIKA 2026 | All rights reserved

Lab 4.4 · 4.6 : le même enseignement

LAB 4.4 · 4.6

Spec Kit enchaîne `specify → plan` sans étape de conception, et rien n'empêche d'y en insérer une.

Visuel ou contractuel, le mécanisme est le même.

© ZENIKA 2026 | All rights reserved

Lab 4.5 · facultatif · Spec Kit perso (3/3)

Rejouer le Pomodoro, puis comparer

La phase de design devient un garde-fou réutilisable, et le wireframe, un critère de recette.

Consignes : handout « Lab 4.4 à 4.6 · Phase de design (facultatif) », étape 4.5.



© ZENIKA 2026 | All rights reserved

Chapitre 5

Mise en pratique sur un projet

Construire en SDD un framework de pilotage de tests, puis re-documenter un legacy.



© ZENIKA 2026 | All rights reserved

Les fondations posées au Ch. 4

Au chapitre précédent, on a vu que **la doc est le patrimoine**, pas le code — et construit les **garde-fous de commit** qui le protègent :

- **SPEC.md comme source de vérité** : spec fonctionnelle et doc technique en un seul fichier — format inspiré d'OpenAI Symphony, consommable directement par l'agent.
- **Extension** `doc-tracker` : traçabilité IA dans chaque commit + vérification de la doc avant de pusher.
- **Gradient de personnalisation** : du framework rigide au workflow maison *opinionated*.

> *i* **Fil rouge** : l'extension `doc-tracker` construite au Ch. 4 sera **branchée sur le serveur E2E** dans le TP de ce chapitre — chaque commit vérifiera que la **documentation est à jour** et que les **logs sont correctement écrits**.

© ZENIKA 2026 | All rights reserved

Ce qui devient possible : l'outil qu'on n'aurait jamais écrit

Tout développeur a une liste d'outils internes qu'il **n'a jamais faits** — pas parce qu'ils sont difficiles, mais parce que le rapport effort / bénéfice ne passait pas :

- le petit tableau de bord qui éviterait de relancer trois commandes à la main ;
- le script qui automatiserait la corvée hebdomadaire ;
- le framework de test qui rendrait enfin les E2E supportables.

Ce calcul vient de changer. Ce qui coûtait deux semaines en coûte deux jours — et l'outil « pas assez rentable » le devient.

> **C'est le sujet du chapitre** : on ne va pas *étudier* un framework de pilotage de contrat d'API ou de tests E2E, on va en **construire un**, en SDD, et repartir avec.

> **Pourquoi** : le flux de travail SDD est différent du flux de développement standard. L'avoir déroulé une première fois sur un ensemble de features permet d'en saisir la portée réelle, et les limites actuelles.

© ZENIKA 2026 | All rights reserved

Tests d’API ou E2E : enfin créables ET maintenables

Hier, ces tests étaient **coûteux** à écrire et **fragiles** : le moindre changement — de **schéma** ou d’**interface** — les cassait → souvent abandonnés.

Aujourd’hui l’IA change la donne sur **tout le cycle**, quel que soit le terrain :

	Tests d’API	Tests E2E
Générer	un test de contrat, depuis la spec OpenAPI	un script de navigation, depuis une description
Exécuter	requêtes / réponses tracées, écarts de schéma	captures d’écran, traces
Diagnostiquer	régression de contrat, ou versionnage assumé ?	vraie régression, ou évolution d’UI ?
Réparer	le test — ou faire évoluer le contrat	le script, si l’intention tient toujours

i Le pilotage : un **moteur de contrat** (Schemathesis, Pact) ou un **navigateur automatisé** (Playwright), et l’IA pour générer, diagnostiquer et réparer.

© ZENIKA 2026 | All rights reserved

Lab 5 · préambule · Pilotage des tests en SDD

Tests d’API ou Tests E2E

Deux terrains, un seul apprentissage : on choisit sa colonne avant de lancer le TP.

On code l’intention, pas les lignes : le moteur s’occupe du reste.



© ZENIKA 2026 | All rights reserved

Lab 5 : ce que doit tenir le framework

LAB 5

- **Dev totalement automatisé** : le code est un **asset régénérable**, tant que la spécification est respectée.
- **Qualité & maintenabilité** au rendez-vous : c'est tout l'objet du framework.
- **Sécurité dès le départ** : authentification de tous les points d'accès, secrets hors du code…
- **Architecture minimale** : Docker / docker-compose, base type SQLite, et on **pilote les décisions**.
- **L'application appelle elle-même un modèle**, pour diagnostiquer un échec et proposer une réparation : elle a sa **propre clé d'API**, distincte de votre assistant. *(Options : handout « Prérequis des labs ».)*
- `doc-tracker` **branché dès le départ** : chaque commit vérifie que la doc suit et trace la part d'IA.

> **Règle minimale** : « régénérable » ne veut pas dire « illisible ». La **lisibilité et la compréhensibilité humaine du code** restent non négociables, quel que soit le niveau de délégation à l'IA : sécurité, auditabilité et conformité l'exigent.

© ZENIKA 2026 | All rights reserved

Lab 5.2 · 5.3 · Pilotage des tests en SDD

Décrire, exécuter, suivre

Objectif : construire une application de pilotage des tests d'API ou des tests E2E, en Spec-Driven Development.

Consignes : handout « Lab 5 · Pilotage des tests en Spec-Driven Development », étape 5.2 (tests E2E) ou 5.3 (variante backend : tests de contrat, tests d'API).



© ZENIKA 2026 | All rights reserved

L'IA débloque les migrations legacy

L'IA **facilite déjà** les migrations **techniques** (montées de version, frameworks), là où l'outillage statique type Renovate / Dependabot ne fait que **détecter**.

Surtout, elle rend **abordables** les migrations d'**applications legacy**, en **tout ou partie** :

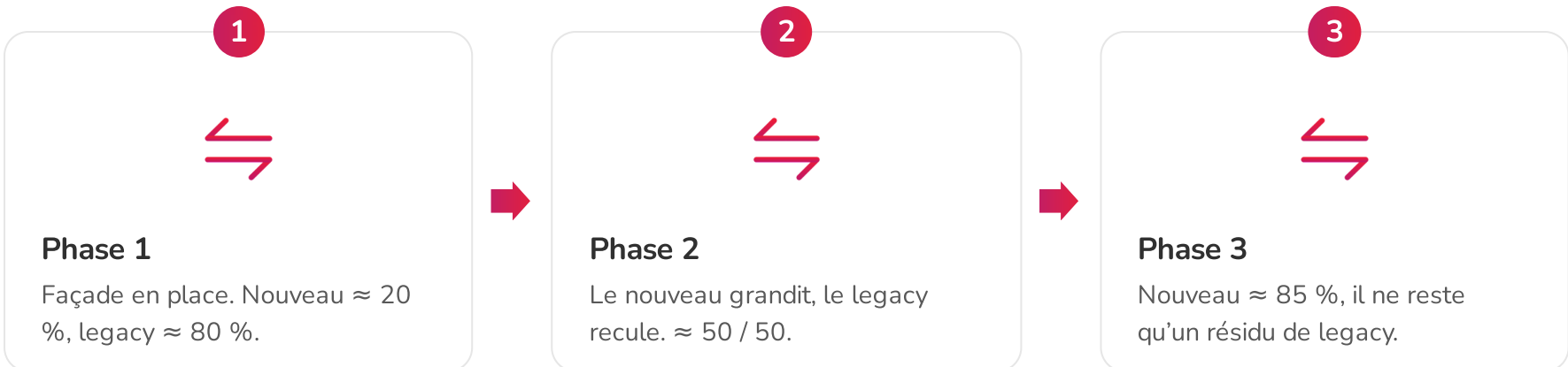
- code **plus vraiment maîtrisé**, voire **difficilement accessible** ;
- documentation **parcellaire**, **pas à jour**, ou **plusieurs versions** coexistent sans savoir **laquelle fait foi**.

> *i* On ne réécrit pas à l'aveugle : **re-documenter d'abord**, **puis** définir une **stratégie de refonte**, *pas forcément d'un coup*.

© ZENIKA 2026 | All rights reserved

Refondre par étapes : le Strangler Fig

Métaphore du **figuier étrangleur** (M. Fowler) : le neuf **enserre** le legacy et le **remplace feature par feature**, derrière une **façade** qui route les requêtes.



Le nouveau enserre puis remplace le legacy, sans big bang.

Intérêt : on **livre en continu**, sans *big bang* ni « branche-tunnel » de réécriture.

© ZENIKA 2026 | All rights reserved

Compléments au Strangler Fig

Pattern	Principe	Intérêt
Branch by Abstraction	une couche d'abstraction ; ancien & nouveau coexistent dans le code	refondre en interne, sans branche longue
Anti-Corruption Layer	un adaptateur qui traduit entre legacy et neuf	protéger le nouveau modèle des sémantiques legacy
Parallel Run	exécuter les deux, comparer les sorties	valider la non-régression avant bascule



Souvent **combinés** : Strangler Fig **à la frontière**, Branch by Abstraction **dans le code** (Fowler ; Sam Newman, *Monolith to Microservices*).

© ZENIKA 2026 | All rights reserved

Re-documenter d'abord

Tous ces patterns supposent de **savoir ce que fait** le système, or c'est justement ce qui **manque** sur un legacy.

- **Rétro-documenter** = reconstruire une **spec fiable** (intention, règles métier, contrats) **à partir du code**.
- Cette spec devient l'**entrée du SDD** → on peut alors **régénérer** (tout ou partie) **en confiance**.

> *i* C'est l'objet du **lab suivant** : *du legacy → specs → régénération*.

© ZENIKA 2026 | All rights reserved

Pourquoi on ne peut pas juste « donner le legacy à l'IA »

L'approche naïve — *tout charger, puis demander de réécrire* — **ne marche pas**, et c'est ce qui structure tout le reste :

- **500 000 lignes de code ≈ plusieurs millions de tokens.** Face à une context window d'**1 M au mieux**, ça ne rentre pas. Et même si ça rentrait, l'attention se dégrade bien avant la limite *(cf. Ch. 2)*.
- Sur quelques **centaines** de lignes, l'IA y arrive en claquant des doigts — mais **vous aussi**. Le besoin réel est entre les deux.
- Il faut donc **attaquer le code par morceaux structurés** : packages, modules, couches — un découpage qui dépend du langage et de l'architecture.
- D'où le rôle du **knowledge graph** *(Ch. 2)* : l'agent **navigue** dans le code au lieu de l'avaler, et ne charge que ce dont il a besoin.

> *i* **Le pivot n'est pas le code, c'est la documentation qu'on en extrait.** On ne transcrit pas ligne à ligne : on reconstruit l'intention, puis on régénère à partir d'elle.

© ZENIKA 2026 | All rights reserved

Lab 6 · Rétro-documentation

Du legacy aux specs, puis on régénère

Objectif : partir d'un projet legacy sans doc, en extraire les spécifications, les injecter dans un outil SDD, puis recréer l'application.

Reversa ne touche pas au code existant (écrit dans `.reversa/` et `_reversa_sdd/`), il documente avant de régénérer.

Consignes : handout « Lab 6 · Rétro-documentation ».



© ZENIKA 2026 | All rights reserved

Sur un projet existant : une transition, pas une bascule

Un projet de dix ans ne passe pas au SDD un lundi matin. La question n'est pas *« quand est-ce qu'on migre ? »* mais *« par quel bout on commence ? »*

Ce qui bloque, honnêtement :

- L'existant impose ses contraintes : on ne peut **pas** se baser uniquement sur les fichiers produits par le framework SDD.
- La documentation complète, celle qui devrait servir de pivot, est **encore peu standardisée** : chaque framework a son format, aucun ne fait autorité.

© ZENIKA 2026 | All rights reserved

Sur un projet existant : les premiers garde-fous

Étape	Ce qu'on pose	Pourquoi en premier
1	Skills et instructions de projet	Valeur immédiate, aucun changement de méthode
2	Constitution : les contraintes non négociables du projet	Cadre tout ce qui sera généré ensuite
3	Specs au sens SDD sur les *nouvelles* features	On ancre la méthode sans toucher à l'existant
4	Rétro-documentation du périmètre qu'on va faire évoluer	Seulement là où on en a besoin



On ne rétro-documente pas tout un legacy « pour être propre » : on le fait sur le périmètre qu'on s'apprête à toucher. Le reste attend son tour, ou n'y vient jamais.

© ZENIKA 2026 | All rights reserved

Chapitre 6

Le futur du dev : coder autrement

Superviser au lieu d'approuver : poste terminal-first, agents en parallèle.



© ZENIKA 2026 | All rights reserved

Fil rouge : monter d’un cran en autonomie

Du **Ch. 1 au Ch. 5**, l’agent a travaillé **sous votre œil** : une session à la fois, un périmètre, vos validations. Au terminal dès le Ch. 1, puis sur un vrai projet au Ch. 5.

Le Ch. 6 pousse jusqu’au bout le curseur des **modes de pilotage** (Ch. 1) :

- l’agent avance **plus longtemps sans vous** : vous **supervisez** au lieu d’approuver.
- le poste devient une **console de supervision** : plusieurs sessions visibles d’un coup d’œil.
- plusieurs agents progressent **en parallèle**, chacun dans son périmètre isolé.

> *i* La question du chapitre : **que devient mon poste de travail quand l’agent n’attend plus mon feu vert à chaque étape ?**

© ZENIKA 2026 | All rights reserved

Superviser plusieurs agents : ce que ça demande au poste

L'IDE est conçu pour une personne qui édite un fichier. Superviser plusieurs agents qui tournent est un autre métier :

Besoin quand l'agent est autonome	IDE-first	Terminal-first
Voir N sessions en même temps	un panneau de chat à la fois	panneaux d'un multiplexeur de terminal côte à côte
L'agent survit à la fermeture de la fenêtre	non	session détachable (multiplexeur de terminal)
Rejouer / scripter ce qu'on vient de faire	à la souris	bash, make, `-p "…"`
Isoler l'exécution (cf. sandboxing, Ch. 1)	poste local	SSH · conteneur · VM
Relire un diff ligne à ligne avant de valider	panneau Source Control	lazygit · `git diff`

i L'agent **ne remarque pas** si vous êtes dans VS Code ou dans un terminal. C'est **votre** posture qui change : de relecteur d'une suggestion à **superviseur de plusieurs chantiers**.

© ZENIKA 2026 | All rights reserved

lazygit — git visuel dans le terminal

lazygit : TUI git — diff, staging, branches, commits — sans quitter le terminal.

```
# Installation
brew install lazygit    # macOS
apt install lazygit     # Debian Ubuntu 22.04+

# Lancer dans le repo
lazygit
```

Fonctionnalités clés :

- **Staging interactif** : sélectionner des **lignes individuelles** à stager (`Space`)
- **Branch graph** : historique des commits visualisé
- **GitHub / GitHub Enterprise** : lien vers les PRs depuis la liste de branches
- **Raccourcis clavier riches** et personnalisables

> *i* **Valeur clé avec les agents** : après qu'un agent ait tourné, `lazygit` permet de **relire précisément ce qu'il a changé**, ligne par ligne, avant de committer ou de rejeter.

© ZENIKA 2026 | All rights reserved

Le poste terminal-first en action

Copilot CLI (agent)

```
$ copilot   (feat/clinic-list, 2 panneaux)
▶ Implémente la page « All Clinics »
  selon la story docs/stories/all-clinics.md
● Lecture de AGENTS.md… ok
● Lecture de la story… ok
● Création de ClinicController.java…
● Création de ClinicRepository.java…
● En attente : ClinicService.java
❯ _
```

lazygit

```
lazygit — feat/clinic-list
○ main    ↑2  ORIG_HEAD 2bdb744
● feat/clinic-list
  M ClinicController.java
  A ClinicRepository.java
  A ClinicService.java
  ? spec.md (untracked)
```

Un seul écran : l'agent génère à gauche, `lazygit` montre le diff à droite. La relecture avant commit ne demande pas de quitter le poste.

© ZENIKA 2026 | All rights reserved

Le mémo de survie d'un CLI agentique

Six besoins reviennent dans tous les labs. Les noms de commandes changent d'un outil à l'autre, les besoins non : exemples ci-dessous avec le CLI de la session.

Besoin, en séance	Exemple *(GitHub Copilot CLI)*
Ouvrir une session sur un dépôt	`copilot` · `/cwd` pour en changer
Mesurer et maîtriser la context window *(Ch. 2)*	`/context` · `/compact [focus on …]` · `/clear`
Arbitrer coût / puissance	`/model`
Déléguer · brancher un outil	`/agent` · `/mcp`
Relire, reprendre, revenir en arrière	`/diff` · `/resume` · `/session`
Tourner sans supervision, mais borné	`-p "…" --allow-tool 'shell(mvn test)'`

© ZENIKA 2026 | All rights reserved

Le seul réflexe non négociable

Une permission ciblée plutôt que toutes les permissions.

En non interactif, ouvrir tous les droits par confort est exactement ce que le chapitre harness interdit.

© ZENIKA 2026 | All rights reserved

Lab 7 · facultatif · Poste terminal-first

Monter sa console de supervision

Lab facultatif, à jouer si le temps le permet. Sinon, la démonstration du slide précédent suffit.

Objectif : mettre en place un poste de supervision (un multiplexeur de terminal au choix + `lazygit`), puis développer une feature en laissant l'agent avancer au lieu de valider chaque étape. Le workflow complet : agent génère → `lazygit` valide → commit.

Consignes : handout « Lab 7 · Poste terminal-first (facultatif) ».



© ZENIKA 2026 | All rights reserved

Développer plusieurs features simultanément

Contexte : deux features indépendantes à livrer, ou simplement **interrompre** une tâche en cours pour un **hotfix** (ou une revue) sans tout *stasher*.

Sans worktrees

```
git stash            # mettre de côté A
git checkout feat/B  # démarrer B
# ... dev B ...
git checkout feat/A  # revenir à A
git stash pop
```

→ Un seul agent actif à la fois, allers-retours coûteux.

Avec worktrees + agents

```
# Deux branches dans deux
# répertoires distincts
git worktree add ../proj-feature-a \
    feat/clinic-list
git worktree add ../proj-feature-b \
    feat/search-filter
# Deux agents tournent en parallèle
```

→ Les deux features avancent **simultanément** ; le temps total = le plus long des deux.

> *i* Pour des features de 30 min chacune : on passe de **60 min** à **~30 min** + temps de merge.

© ZENIKA 2026 | All rights reserved

Git worktrees — le mécanisme

Un **worktree** = un répertoire de travail supplémentaire attaché au **même dépôt git** (même base d'objets, historique partagé, mais HEAD et index indépendants).

Git natif : marche avec **tout** agent, et sert aussi au context-switch sans `stash`.

```
git worktree add -b feat/x ../proj-x   # nouvelle branche + dossier dédié
git worktree list                       # worktrees actifs
copilot                                 # on lance l'agent DANS ce dossier
git worktree remove ../proj-x           # nettoyage après merge
```

> *i* Les fichiers non versionnés ne suivent pas : `.env`, secrets, `node_modules/` sont absents du nouveau worktree. À copier (ou à scripter) sinon l'agent démarre sur un projet qui ne compile pas.

© ZENIKA 2026 | All rights reserved

Worktrees : ce que le harness automatise (ou pas)

C'est **très inégal** d'un agent à l'autre, et ça bouge tous les trimestres.

Harness	Worktree géré par l'outil	En pratique
Copilot	✗ en CLI *(→ skill communautaire)* · ✓ app desktop : 1 worktree isolé par session parallèle	git en CLI, automatique dans l'app
Claude Code	✓ `--worktree` · fichier de règles pour embarquer les fichiers ignorés	l'agent crée le worktree et démarre dedans
Codex	✗ en CLI · ✓ app desktop (mode *Worktree*) · sous-agents isolés	git en CLI
OpenCode · Aider	✗	git, éventuellement via un runner tiers

> **Ce qui est portable, c'est git.** L'automatisation est un confort, présent chez certains seulement : ne bâtissez pas le workflow de l'équipe sur une fonctionnalité qu'un seul outil possède.

© ZENIKA 2026 | All rights reserved

Pas de feature native ≠ tout à la main

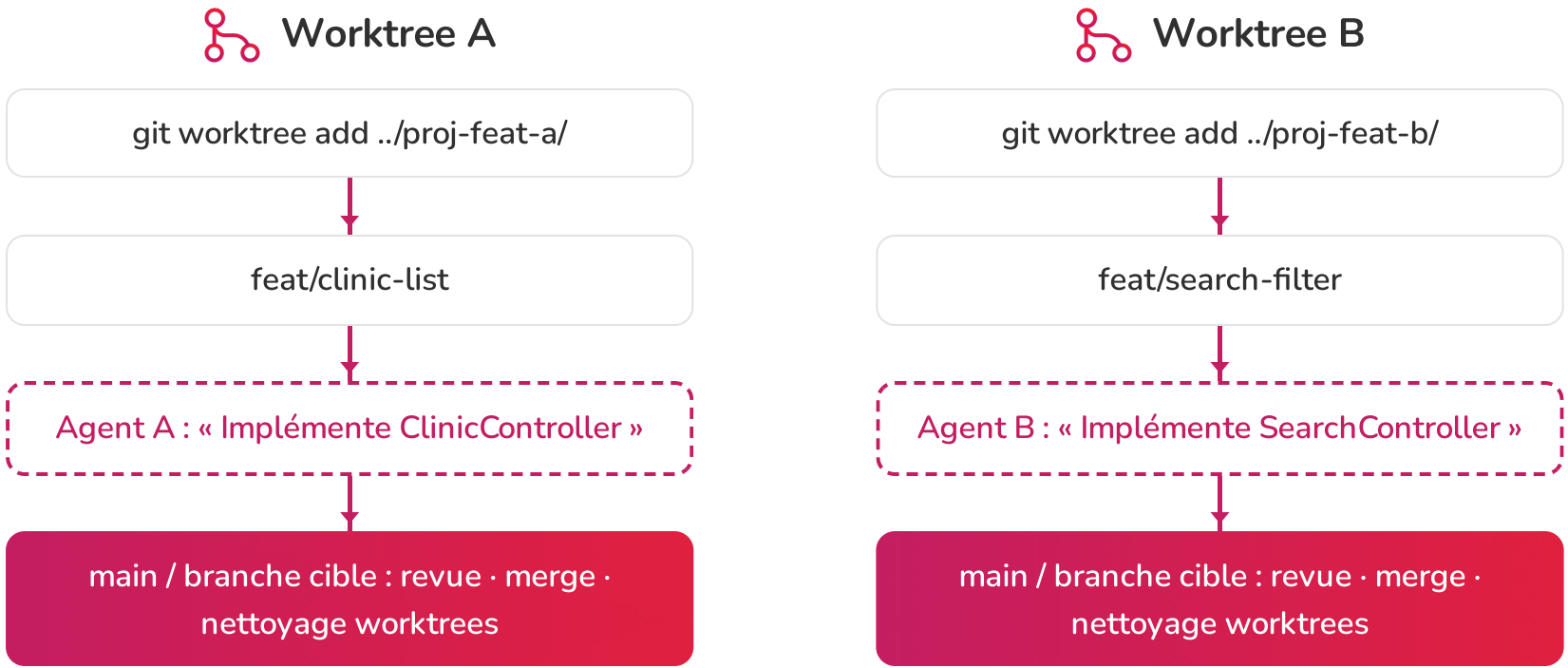
Là où le harness ne fournit rien, un **skill de gestion de worktrees** comble le manque.

C’est exactement à ça que sert un harness **extensible**.

© ZENIKA 2026 | All rights reserved

Agents en parallèle sur worktrees

Un **dépôt git** (object database partagée : `.git/`, historique, objets), deux worktrees qui **tournent en même temps**.



© ZENIKA 2026 | All rights reserved

Tradeoffs du développement parallèle

Avantage	Coût / risque
Temps divisé : deux features avancent simultanément	Tokens × n : chaque agent a son propre contexte
Contextes isolés : les agents ne se perturbent pas	Conflits de merge : si les features touchent les mêmes fichiers
PRs plus petites : revue plus lisible et rapide	Votre attention : au-delà de 2 sessions, on ne supervise plus, on subit
Qualité améliorée : contexte ciblé → moins d'hallucinations	Services : conflits de ports (DB, Redis) si les features partagent l'infra

i Le parallélisme **divise le temps machine** mais **multiplie les ressources** — et le facteur qui sature en premier n'est ni le disque ni les tokens : c'est **vous**.

© ZENIKA 2026 | All rights reserved

Quand utiliser le développement parallèle ?

Quand ça vaut le coup :

- Features **vraiment indépendantes** (modules ou composants distincts)
- Durée estimée **> 15–20 min** d'agent chacune
- Équipe à l'aise avec le processus de merge

Cas où les worktrees sont déconseillés :

- **Monorepos avec services partagés** (multiple BDs, ports) → GitButler (branches virtuelles, un seul worktree) est une alternative confirmée en 2026
- **Dépendances très lourdes** → `npm install` × n à chaque worktree

> *METR (juil. 2025, 16 développeurs, 246 tâches)* : les développeurs avec IA ont pris **19% plus longtemps**, non moins. L'IA aide, mais pas automatiquement — la discipline de revue et la qualité des instructions restent déterminantes.

© ZENIKA 2026 | All rights reserved

Le coût caché : l'érosion de la connaissance

Monter en autonomie a une contrepartie dont on parle peu. Elle n'est ni technique ni budgétaire :

- **Le code vous appartient moins** quand vous ne l'écrivez pas — et **encore moins** quand vous vous contentez de le relire.
- **La relecture n'est pas de l'apprentissage** : on valide une forme, on n'acquiert pas le raisonnement qui y a mené.
- **L'effet est lent et invisible** : personne ne constate une perte de compétence à la fin du sprint. On la constate en incident de production, deux ans plus tard.
- **L'IA amplifie ce que vous êtes déjà** : bonne base de code → très bonne ; mauvaise → bien pire. Tendance à se disperser → dispersion multipliée.

> *i* **Ce n'est pas un argument contre l'autonomie, c'est sa condition** : ce qu'on automatise, il faut savoir le **reprendre**. D'où la doc et la spec comme patrimoine — le fil rouge du module.

> *i* La réponse concrète est déjà dans ce que vous avez construit : specs versionnées, `SPEC.md` à jour, ADR, traçabilité au commit. **La documentation est ce qui reste quand la mémoire de l'équipe s'efface.**

© ZENIKA 2026 | All rights reserved

Faut-il arrêter la revue de code ?

Certaines équipes la suppriment déjà : bots de revue en CI, et pour le reste *« relis-le avec ton assistant »*. **L'IA qui relit l'IA.** Notre position :

La revue doit évoluer. Relire ligne à ligne du code qu'on n'a pas écrit n'a plus le même rendement. Le curseur se déplace vers les **tests qui portent une vraie valeur** : ceux qui décrivent le comportement attendu, pas ceux qui paraphrasent l'implémentation.

Mais on ne la supprime pas. C'est une décision qui se pèse, pas un défaut de configuration. Et le message « l'IA relit l'IA, c'est réglé » n'est pas le bon.

© ZENIKA 2026 | All rights reserved

Revue de code : la conviction Zenika

Conviction Zenika, en l'état de nos retours : tant qu'on n'a pas **un à deux ans de recul**, avec des **incidents de production réellement résolus sans qu'un humain touche au code**, on **n'arrête pas les revues**. Les coupures franches ne peuvent pas faire partie de nos conseils : nous manquons tous honnêtement de recul. Restons pragmatiques.

La vraie priorité est ailleurs : renforcer une **documentation lisible par les humains *et* par les agents**, sur tous les aspects du produit. Tant qu'elle n'existe pas, on n'a **aucune garantie** sur ce qui est produit : les tests n'en sont qu'un complément.

© ZENIKA 2026 | All rights reserved

Lab 8 · Développement parallèle

Deux features en parallèle

Objectif : développer deux features indépendantes de la PetClinic en parallèle avec deux agents sur deux worktrees, puis les réconcilier.

Comparer le temps total avec une approche séquentielle estimée. Identifier les fichiers qui auraient conflicté.

Consignes : handout « Lab 8 · Développement parallèle ».



© ZENIKA 2026 | All rights reserved

Ce qui diffère vraiment entre harness

Après cinq chapitres de pratique, la liste honnête. Six écarts ne se réduisent pas à un renommage de chemin :

Écart	Ce qui change réellement	Vu au
Déclencher un skill	modèle seul · `/<nom>` · `$<nom>` selon l'outil	Ch. 1
Context window vs dépense	endroits et unités différents, aucune équivalence directe	Ch. 2
Mémoire	aucun format commun : rien ne se transfère d'un agent à l'autre	Ch. 2
Modèle & fournisseur	3 postures d'écosystème : décision d'achat et de conformité	Ch. 1
Ancrage forge	droits MCP pré-câblés, agent cloud, revue de PR : continuité inégale	Ch. 1 · M3
Automatisation worktree	présente, absente, ou réservée à l'app desktop	Ch. 6

© ZENIKA 2026 | All rights reserved

Portable, et ce qui doit être écrit

Ce qui est portable : `AGENTS.md`, `SKILL.md`, MCP, le principe des hooks, et surtout la **méthode**. Changer d'agent coûte un renommage sur ces quatre-là.

Ce qui ne l'est pas doit être écrit. Un écart d'outil non documenté devient une divergence silencieuse dans l'équipe : c'est exactement ce que le Ch. 7 va outiller.

© ZENIKA 2026 | All rights reserved

Coder autrement : piloter une session à distance

L'agent tourne sur une machine, on interagit depuis un autre poste ou un smartphone.

Harness	Mécanisme	Détail
Claude Code	`/remote-control`	session en cours, prise de main depuis un autre terminal ou l'app mobile Claude
Codex (OpenAI)	app mobile ChatGPT	setup par QR code, envoie prompts et approvals depuis le mobile, supporte SSH vers hôtes distants
OpenCode	`opencode web` + `opencode attach`	serveur web local, TUI distant. App de remote control expérimentale (communautaire)
Copilot CLI	`copilot --cloud`	sandbox cloud, reprise depuis une autre machine (pas du remote control d'une session locale)
Devin	`/handoff`	transfert d'une tâche locale vers Devin Cloud, accessible depuis app.devin.ai

© ZENIKA 2026 | All rights reserved

Session à distance : le cas d’usage

Lancer un long traitement **avant de partir**, vérifier l’avancement **depuis son téléphone**, **intervenir** si l’agent dérive.

L’agent n’est plus attaché au terminal ouvert.

© ZENIKA 2026 | All rights reserved

Coder autrement : vérifier visuellement le résultat

L'agent ouvre et manipule un navigateur pour **voir** ce qu'il a produit (page web, UI).

Harness	Capacité navigateur
Claude Code	extension Chrome : navigation, clic, saisie, captures d'écran, lecture du DOM et des logs console
Codex (OpenAI)	navigateur intégré à l'app desktop : navigation, captures, Developer mode avec accès au réseau et à la console
Cursor	outil Browser natif : navigation, clic, captures, lecture console + trafic réseau, persistence des cookies
Devin	Interactive Browser : navigation, captures, vidéos, intervention humaine pour CAPTCHA/MFA

i L'agent ne se contente plus de générer du code et de faire tourner les tests : il **voit** le résultat, comme un développeur qui ouvre son navigateur pour checker son travail. La boucle « coder → tester → regarder » devient autonome.

© ZENIKA 2026 | All rights reserved

Synthèse : superviser au lieu d'approuver

Deux moyens **au service du même saut d'autonomie**, adoptables séparément :

Le poste comme console de supervision

- **Multiplexeur de terminal** (au choix) · `lazygit` → sessions persistantes, relecture de diff sans quitter le terminal
- Un **CLI agentique** dans chaque panneau → on voit ce que l'agent fait, on ne le devine pas
- Valeur : persistance, scriptabilité, relecture ligne à ligne avant de valider

Le parallélisme, quand les périmètres sont vraiment indépendants

- **Git worktrees** → isolation des branches sans dupliquer le dépôt
- **Agents concurrents** → chacun son contexte, sans interférence
- Valeur : temps divisé, PRs plus petites, contextes ciblés

i Ce qui rend l'autonomie tenable, ce n'est pas l'outillage : ce sont les **tests, les commits fréquents et la relecture du diff**. Sans ce filet, monter d'un cran ne fait qu'accélérer la production de code non vérifié. La prochaine étape : faire passer ces pratiques **dans l'équipe**, c'est l'objet du Ch. 7.

© ZENIKA 2026 | All rights reserved

Chapitre 7

Partage & mutualisation en équipe

Comment l'équipe trouve, fait évoluer et fait confiance à ses skills, agents et instructions.



© ZENIKA 2026 | All rights reserved

Fil rouge : de l'individuel au collectif

Depuis le Ch. 1, vous avez construit des **assets de valeur** :

- **AGENTS.md** personnalisé (Ch. 1) — contexte de projet, règles, conventions
- **Skills** (Ch. 1, 4) — expertise métier encodée, réutilisable
- **Workflows SDD** (Ch. 3–5) — pipelines spec → tests → migration
- **Agents spécialisés** (Ch. 1) — code review, front, back, tests

Ces assets ont une **valeur individuelle** — et une **valeur d'équipe décuplée** :

- Éviter la **duplication** (dix développeurs, dix AGENTS.md qui redisent la même stack)
- Capitaliser sur les **découvertes** d'un développeur → profiter à toute l'équipe
- Maintenir la **cohérence** des pratiques entre membres

> *i* La question clé : **comment l'équipe trouve, fait évoluer et fait confiance à ces assets ?**

© ZENIKA 2026 | All rights reserved

Partager les skills au niveau projet

Les skills dans `.github/skills/` sont **versionnés avec le code** : tout développeur qui clone le repo récupère automatiquement les skills de l'équipe.

```
.github/
├── skills/
│   ├── reviewing-prs/SKILL.md            # skill de revue de PRs
│   ├── generating-specs/SKILL.md         # skill de génération de specs SDD
│   ├── analysing-e2e-failures/SKILL.md   # skill diagnostic E2E (Ch. 5)
│   └── documenting-api/SKILL.md          # skill doc technique
├── agents/                               # agents custom (Ch. 1)
└── hooks/                                # automatisations (Ch. 1)
```

© ZENIKA 2026 | All rights reserved

Partager les skills : les scopes

Scope	Chemin	Partagé via
Projet	`.github/skills/` (aussi `.claude/skills/`, `.agents/skills/`)	git (automatique)
Personnel global	`~/.copilot/skills/` · `~/.agents/skills/`	non partagé
Organisation	skills & instructions d'org. (Business / Enterprise)	plateforme GitHub



Les skills construits aux Ch. 1, 4 et 5 trouvent ici leur **home naturel**, et sont consommés par **toutes** les surfaces : CLI, IDE, agent cloud, code review.

© ZENIKA 2026 | All rights reserved

Bonnes pratiques de skills (1/2)

D'après la spécification ouverte Agent Skills et les docs GitHub / Anthropic.

Nommage : forme **gérondive**, descriptif de l'action. Lettres minuscules, chiffres, tirets ; max 64 caractères.

- ✓ `reviewing-prs` · `generating-specs` · `analysing-logs`
- ✗ `my-skill` · `helper` · `anthropic-review` *(mot réservé)*

Description (max 1024 car.) : à la **troisième personne**, en incluant **ce que ça fait** ET **quand l'utiliser** → meilleure activation automatique. *« Use this skill when a developer asks to review a pull request… »*

> *i* Sur Copilot, l'activation est **décidée par le modèle** à partir de la `description`. Une description vague = un skill qui ne se déclenche jamais. C'est la ligne la plus importante du fichier.

© ZENIKA 2026 | All rights reserved

Bonnes pratiques de skills (2/2)

Corps du skill (max 500 lignes) :

- Ne donner que ce que l'agent **ne peut pas déduire seul**
- Éviter les chemins spécifiques à un OS, les versions figées, les informations périmées

Test d'autonomie « agent A → agent B » :

- **Agent A** écrit le skill
- **Agent B** (session fraîche, sans contexte) le teste sans coaching
- Si l'agent B l'utilise correctement → le skill est **autonome**

© ZENIKA 2026 | All rights reserved

Distribuer ses skills : `gh skill` & skills.sh

Un skill est un dossier dans un dépôt : il se distribue donc **comme une dépendance**, avec version et provenance.

```
# gh skill (GitHub CLI ⩾ 2.90, avr. 2026) : la voie GitHub-native
gh skill search "code review"
# explorer le dépôt
gh skill install github/awesome-copilot
# un skill précis
gh skill install github/awesome-copilot documentation-writer
# épinglé sur un tag
gh skill install my-company/dev-skills reviewing-prs@v1.2.0

# skills.sh (Vercel, jan. 2026) : l'annuaire public, multi-agents
npx skills search "code review" && npx skills add vercel-labs/agent-skills
```

© ZENIKA 2026 | All rights reserved

Distribuer ses skills : provenance, épinglage, sécurité

- **Provenance tracée** : `gh skill` inscrit dépôt, ref et SHA dans le frontmatter du `SKILL.md` installé.
- **Épinglage** : par tag ou par commit, donc **reproductible** pour toute l'équipe.
- **Multi-agents** : les deux outils posent des skills utilisables par Copilot, Claude Code, Cursor, Codex, Gemini CLI…
- **Sécurité** : scan **Snyk** automatique à l'installation côté skills.sh.

> *i* Un skill installé, c'est **du prompt exécuté chez vous** (et parfois des scripts). À revoir comme n'importe quelle dépendance tierce.

© ZENIKA 2026 | All rights reserved

La marketplace GitHub : un repo comme registre

Pattern : un **dépôt GitHub dédié aux skills** déclaré comme source → l'agent y accède automatiquement.

```
# Structure d'un repo de skills d'équipe
my-company/dev-skills/
├── README.md                    # catalogue + guide d'utilisation
├── reviewing-prs/
│   └── SKILL.md
├── generating-specs/
│   └── SKILL.md
├── analysing-e2e-failures/
│   └── SKILL.md
└── CODEOWNERS                   # qui maintient quoi
```

Le pattern d'équipe : un dépôt interne de skills + `CODEOWNERS` + tags de version, consommé par `gh skill install my-company/dev-skills <skill>@<tag>`. Une marketplace privée sans plateforme à déployer.

© ZENIKA 2026 | All rights reserved

Registres confirmés en 2026

- **github/awesome-copilot** : skills, agents et instructions, installables par `gh skill install`
- **VoltAgent/awesome-agent-skills** : 1000+ skills multi-agents (github.com/VoltAgent/awesome-agent-skills)
- **anthropics/skills** : skills de référence, validés, documentés, versionnés (format identique)
- **MCP Registry** (`registry.modelcontextprotocol.io`) : 9 652 serveurs MCP, lancé sept. 2025, gouverné par l'**Agentic AI Foundation (AAIF)** (Linux Foundation, depuis déc. 2025)
- **skills.sh** : annuaire public avec qualité auditée + Snyk

© ZENIKA 2026 | All rights reserved

Plugins & marketplaces : un cran au-dessus du skill

Un **skill** est une unité (un dossier, une capacité). Un **plugin** en empaquette **plusieurs** — skills, hooks, agents custom, commandes — sous un seul nom installable. Une **marketplace** n'est que la couche de distribution de l'un ou l'autre.

```
# Déclarer une marketplace (un dépôt GitHub, une URL git, ou un chemin local)
/plugin marketplace add <org>/<repo>

# Installer un plugin qu'elle référence
/plugin install <nom-du-plugin>@<nom-de-la-marketplace>
```

- Le plugin est décrit par un manifeste `.claude-plugin/plugin.json` (nom, version, auteur) à côté de ses composants.
- Une marketplace n'est qu'un manifeste que l'agent va chercher — un dépôt d'équipe peut en devenir une aussi facilement qu'un dépôt public.

> *i* **Où ça se situe dans le vocabulaire déjà vu :** `AGENTS.md` (règles) < skill (une capacité) < **plugin** (un paquet de capacités) < marketplace (le registre qui les distribue). Chaque niveau peut être versionné et partagé indépendamment des autres.

© ZENIKA 2026 | All rights reserved

Gouvernance : fiabilité sur la durée

Un skill non maintenu est **pire qu'absent** : il génère des comportements incorrects en silence.

Cycle de vie d'un skill d'équipe :

1



Création

Écrire + tester (test agent A→B)

2



Revue

PR + expert domaine, CODEOWNERS

3



Publication

merge + tag sémantique v1.0.0

4



Maintenance

issues · updates · métriques d'usage

5



Dépréciation

deprecated: true + message de remplacement

© ZENIKA 2026 | All rights reserved

Gouvernance : les signaux d’alerte

- Skill utilisé ≠ skill efficace → surveiller les **retours des agents** (réponses inattendues)
- Nouvelle version de l’agent ou changement de modèle → retester les skills critiques
- Convention de nommage de l’équipe change → mettre à jour le corps du skill

© ZENIKA 2026 | All rights reserved

« Ça vaut le coup d'investir ? » : la question qui bloque

Deux objections reviennent systématiquement, et elles sont **légitimes** :

1. « Un skill sera périmé dans six mois. » *(ou : absorbé par le prochain modèle)*
Vrai pour un skill qui **compense une faiblesse du modèle** : reformulation, style, rappel d'une syntaxe. Faux pour un skill qui encode **votre contexte** : conventions maison, procédure de release, règles métier. Aucun modèle ne devinera jamais ça.

2. « La facturation a changé, on nous a dit de lever le pied. » Un changement de tarification refroidit l'investissement, sauf que les skills sont précisément ce qui **réduit** la consommation : moins de répétition, moins d'allers-retours, contexte chargé à la demande.

© ZENIKA 2026 | All rights reserved

Investir ou attendre : le test à trois questions

Investir	Attendre
Contexte spécifique à l'équipe ou au produit	Ce que le modèle fera bientôt nativement
Procédures répétées plusieurs fois par semaine	Le geste qu'on fait deux fois par an
Ce qui a un propriétaire identifié	Ce que personne ne maintiendra

i **Le test à trois questions** : est-ce **spécifique** à nous ? est-ce **répété** ? y a-t-il **quelqu'un pour le maintenir** ? Trois oui → on écrit le skill. Un non → on attend.

© ZENIKA 2026 | All rights reserved

Lab 9 · Skill d'équipe

Créer, tester et partager un skill

Objectif : produire un skill de qualité publication et le partager dans un registre simulé.

Un skill est prêt quand une session fraîche l'utilise sans coaching, pas avant.

Consignes : handout « Lab 9 · Skill d'équipe ».



© ZENIKA 2026 | All rights reserved

AGENTS.md d'équipe : gouvernance des instructions

L'AGENTS.md est passé de **fichier personnel** (Ch. 1) à **fichier d'équipe** — mais qui le maintient, et ce qu'il doit (ou ne doit pas) contenir change.

Quatre couches à séparer :

Fichier	Contenu	Maintenu par
`~/.copilot/`	Préférences perso (style, outils locaux)	Développeur
`./AGENTS.md`	Standards projet (stack, conventions, sécurité)	Tech Lead / équipe
`.github/instructions/*.md`	Règles ciblées par glob (`applyTo`)	Équipe / expert
`.github/skills/<nom>/`	Expertise métier ou technique spécialisée	Expert domaine

i L'AGENTS.md dit **où pointer** ; les skills disent **comment faire**.

© ZENIKA 2026 | All rights reserved

AGENTS.md d’équipe : ce qu’on y met

Bonnes pratiques pour l’AGENTS.md projet :

- **Stack et conventions** : « NestJS 11, TypeScript strict, jest + supertest »
- **Interdits** : « Ne jamais modifier le schéma de DB sans migration »
- **Référence aux skills** plutôt que duplication de leur contenu

> *i* **Règle minimale** : les **instructions de développement partagées et versionnées** (AGENTS.md projet) forment un socle commun prescriptif. Une équipe l’étend, elle ne le contredit pas silencieusement : tout écart est **documenté explicitement**, jamais tacite.

> *i* **Le réflexe de démarrage de projet** : passer une heure en équipe sur le contenu de l’ `AGENTS.md` **avant** d’écrire du code. Tout le monde doit savoir ce qu’il contient, sinon ce n’est plus un socle, c’est un fichier que personne ne relit.

© ZENIKA 2026 | All rights reserved

Auditer ses propres pratiques : `/insights`

On a mesuré la **context window** (Ch. 2) et la **dépense** (Ch. 2). Reste la question qu'aucune des deux ne couvre : *comment est-ce que je travaille, en fait ?*

Côté Claude Code — `/insights` : analyse **vos sessions locales des 30 derniers jours** et produit un **rapport HTML** :

- les **projets** sur lesquels vous passez réellement du temps, et combien de sessions chacun ;
- vos **points de friction**, catégorisés, avec des exemples tirés de vos propres sessions ;
- des **règles** `CLAUDE.md` **proposées** à partir des consignes que vous répétez de session en session ;
- des **pistes non explorées** (workflows autonomes, agents parallèles…), avec un prompt prêt à essayer.

> *i* Tout est **local** : aucune donnée ne sort, l'analyse porte sur ce qui est déjà sur votre machine.

© ZENIKA 2026 | All rights reserved

`/insights` : l'asymétrie et le complément

Côté Copilot, pas d'équivalent côté développeur : `/usage` et `/context` mesurent l'instant, pas la pratique. En revanche l'**organisation** dispose de rapports d'usage par utilisateur, y compris CLI. La symétrie inverse de Claude Code : l'un outille l'**introspection du développeur** (local, rien ne remonte), l'autre le **pilotage de l'organisation**. Aucun des deux n'est complet.

Un outil tiers comble l'écart, tous éditeurs confondus : `agentlytics` (`npx agentlytics`) lit l'historique de plusieurs éditeurs (Cursor, Devin Desktop, Claude Code, Copilot, Zed) et compare l'usage par outil, projet et mois. Sans rien envoyer hors du poste. Complète ccusage (Ch. 2, angle coût) sur l'angle pratique et habitudes.

Prolongement Module 3 : ce que `/insights` fait pour une personne, une passerelle d'entreprise le fait pour une organisation (métriques agrégées, coût par équipe, taux de déclenchement des skills). Le même besoin à deux échelles.

© ZENIKA 2026 | All rights reserved

À l’échelle de l’entreprise

Quand le nombre de skills et d’équipes dépasse une dizaine, la gouvernance manuelle ne suffit plus.

Outillage de niveau entreprise :

- AI Gateway : proxy entre les agents et les fournisseurs de modèles
  - Contrôle des coûts, logging, filtrage de requêtes, failover modèles, registre de serveurs MCP
  - Se branche via le BYOM de Copilot, ou via la variable d’URL de base des autres CLI
  - Exemple open source : Agent Gateway
- Portail interne de skills : catalogue des skills disponibles, guide d’adoption, bonnes pratiques
  - Peut s’intégrer dans Backstage (CNCF) via un plugin custom
  - Pattern : liste catégorisée + auteur + date de dernière revue + score d’usage

© ZENIKA 2026 | All rights reserved

À l'échelle de l'entreprise : repo, politique, métriques

- **Repo central** : `my-company/agent-skills`
  - Tags sémantiques, CODEOWNERS, CI qui vérifie la structure des SKILL.md
  - Consommé par `gh skill install …@<tag>` ; **skills.sh** peut auditer le repo
- **Politique de harness imposée** : hooks au niveau système, que le développeur ne peut pas désactiver
- **Métriques** : combien de fois chaque skill est activé, par quelle équipe → prioriser la maintenance

© ZENIKA 2026 | All rights reserved

Bilan Module 2 : ce que vous avez construit

À l'issue de ce module, vous êtes capables de :

- **Orchestrer** des agents personnalisés (front, back, tests, review) avec `AGENTS.md`, Agent Skills, Hooks et MCP, et **mesurer** ce qui remplit votre context window
- **Spécifier** vos features en Spec-Driven Development (OpenSpec, Spec Kit, BMAD) : spec comme source de vérité
- **Personnaliser** un framework SDD pour votre entreprise (extensions, presets, skills from scratch)
- **Piloter** les tests E2E avec un framework maison (description → script → exécution → réparation auto)
- **Re-documenter** un legacy et le migrer par étapes (Strangler Fig, Reversa)
- **Travailler en terminal-first** et en parallèle (multiplexeur de terminal · lazygit · git worktrees)
- **Partager** vos skills en équipe avec gouvernance (`gh skill`, dépôt catalogué, cycle de vie)

> **Le fil rouge** : *la spec (et la doc) est le patrimoine*, le code est un artefact régénérable. **Et sur un autre outil ?** Tout tient dans des fichiers au format **standard** : changer de harness coûte un renommage, pas une formation.

© ZENIKA 2026 | All rights reserved

Module 3 : piloter la chaîne de production

Tout ce que vous avez construit sur le poste va maintenant entrer dans la chaîne de production.

Module 2 : sur le poste		Module 3 : chaîne de production
CH. 1 *Garde-fous déterministes : hooks*	→	CI *Revue agentique en CI sur chaque PR*
CH. 5 *Tests E2E pilotés manuellement*	→	CI *Planifiés en CI (nuit / push)*
CH. 3–4 *Spec SDD · AGENTS.md · Skills*	→	PDLC *Contrat du PDLC (Discover → Deploy)*
CH. 6 *Worktrees + agents en parallèle*	→	MERGE *PR agents → guardrails avant merge*
CH. 7 *Skills partagés au niveau projet*	→	PORTAIL *Backstage · portail entreprise*

M2 : on touche encore au code, on pilote sur le poste. M3 : on ne touche presque plus au code, on pilote le cycle.

Module 3, *Piloter le développement dans la chaîne de production* : Discover → Design → Develop → Deploy → Deepen. On ne touche **presque plus** au code : on pilote.

© ZENIKA 2026 | All rights reserved

Des questions ?

La spec (et la doc) est le patrimoine, le code est un artefact régénérable.

© ZENIKA 2026 | All rights reserved